

Hybrid Ensemble Learning for Real-Time Phishing URL Detection

CAPSTONE PROJECT REPORT

Submitted by

Venkatesh V T - 99220041774

Sakthi Murugan S - 99220042014

Thivakar V - 99220041953

Hari Sankar T U- 99220041778

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING



SCHOOL OF COMPUTING

COMPUTER SCIENCE AND ENGINEERING

KALASALINGAM ACADEMY OF RESEARCH

AND EDUCATION

KRISHNANKOIL 626 126

April 2026

DECLARATION

We affirm that the project work titled "**HYBRID ENSEMBLE LEARNING FOR REAL-TIME PHISHING URL DETECTION**" being submitted in partial fulfillment for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** is the original work carried out by us. It has not formed part of any other project work submitted for the award of any degree or diploma, either in this or any other University.

Venkatesh V T
[99220041774]

Sakthi Murugan S
[99220042014]

Thivakar V
[99220041953]

Hari Sankar T U
[99220041778]

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

Signature of supervisor

Vimal Subramanian G

Assistant Professor

Department of Computer Science and Engineering



KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)

Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade



BONAFIDE CERTIFICATE

Certified that this project report "**HYBRID ENSEMBLE LEARNING FOR REAL-TIME PHISHING URL DETECTION**" is the bonafide work of "**VENKATESH V T (9920041774), SAKTHI MURUGAN S (9920042014), THIVAKAR V (99220041953), HARI SANKAR T U (99220041778)**" who carried out the project work under my supervision.

Vimal Subramanian G
SUPERVISOR

Assistant Professor

Computer Science and Engineering

Kalasalingam Academy of Research and
Education

Krishnankoil 626126

Virudhunagar District.

Dr. R. Raja Subramanian

HEAD OF THE DEPARTMENT

Associate Professor & Head

Computer Science and Engineering

Kalasalingam Academy of Research and
Education

Krishnankoil 626126

Virudhunagar District.

Submitted for the Project Viva-voce examination held on

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

We would like to begin by expressing our heartfelt gratitude to the Supreme Power for the immense grace that enabled us to complete this project.

We are deeply grateful to the late "**Kalvivallal**" **Thiru T. Kalasalingam**, Chairman of the Kalasalingam Group of Institutions, and to "**Illayavallal**" **Dr. K. Sridharan**, Chancellor, as well as **Dr. S. Shasi Anand**, Vice President, who has been a guiding light in all our university's endeavours.

Our sincere thanks go to our Vice Chancellor, **Dr. S. Narayanan**, for his inspiring leadership, guidance, and for instilling in us the strength and enthusiasm to work towards our goals.

We would like to express our sincere appreciation to **Dr. P. Deepa Lakshmi**, Professor & Dean-(SoC), Director Accreditation & Ranking, for her valuable guidance. Our heartfelt gratitude also goes to our esteemed Head of Department, **Dr. R.Raja Subramanian**, whose unwavering support has been crucial to the successful advancement of our project.

We are especially thankful to our Project Supervisor, **Mr. G.Vimal Subramanian**, for his patience, motivation, enthusiasm, and vast knowledge, which greatly supported us throughout this work.

Our sincere gratitude also goes to **Dr.T.Manikumar** and **Mr.N.R.Sathiskumar** Overall Project Coordinators, for their constant encouragement and support in completing this Capstone Project.

Finally, we would like to thank our parents, faculty, non-teaching staff, and friends for their unwavering moral support throughout this journey.



KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
 Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade



SCHOOL OF COMPUTING
COMPUTER SCIENCE AND ENGINEERING
PROJECT SUMMARY

Project Title	Hybrid Ensemble Learning for Real-Time Phishing URL Detection	
Project Team Members (Name with Register No)	Venkatesh V T – [99220041774] Sakthi Murugan S – [99220042014] Thivakar V – [99220041953] Hari Sankar T U – [99220041778]	
Guide Name/Designation	Vimal Subramanian G	
Program Concentration Area	Machine Learning, Cybersecurity, Ensemble Learning	
Technical Requirements	Python, Machine Learning Libraries, Feature Extraction	
Engineering standards and realistic constraints in these areas		
Area	Codes & Standards / Realistic Constraints	Tick ✓
Economic	The system utilizes open-source machine learning libraries and standard computing hardware, eliminating software licensing costs. Scalable deployment from browser extensions to enterprise systems allows organizations to match investment to their specific needs, with significant ROI through prevented phishing attacks.	✓
Environmental	URL-only analysis minimizes network traffic and computational overhead, consuming minimal energy per URL. The software-only design eliminates electronic waste associated with dedicated hardware appliances and runs on existing infrastructure without additional environmental burden	✓
Social	Protects users from financial fraud, identity theft, and psychological distress associated with phishing attacks.	

	Explainable AI features educate users about phishing indicators, contributing to digital literacy and empowering.	✓
Ethical	Respects user privacy by analyzing only URL strings without accessing webpage content or personal browsing data. Balanced training data ensures unbiased detection across diverse URL types, while transparent decision-making and user override capability respect user autonomy.	✓
Health and Safety	Prevents psychological distress, financial loss, and identity theft resulting from successful phishing attacks. Protects access to healthcare portals and critical services while operating without physical health risks. Aligns with GDPR, HIPAA, and NIST cybersecurity framework requirements	✓
Manufacturability	Implemented entirely in software with modular architecture supporting easy installation and configuration. Deployed through browser extension stores, package managers, and containerized platforms with continuous delivery pipelines for rapid updates and maintenance.	✓
Sustainability	Modular architecture supports continuous improvement and adaptation without complete system replacement. Open-source licensing ensures community maintenance and prevents technology abandonment. URL-only analysis minimizes energy consumption compared to content-based alternatives	✓

REALISTIC CONSTRAINTS:

Economic

The system is designed for cost-effective deployment utilizing open-source machine learning libraries and standard computing hardware, eliminating software licensing costs and specialized equipment requirements. Scalable deployment options from individual browser extensions to enterprise-level security systems allow organizations to match investment to their specific needs. The modular architecture supports incremental adoption, enabling organizations to begin with minimal investment and expand as benefits are realized. Cost-benefit analysis demonstrates significant return on investment through prevented phishing attacks, reduced incident response costs, and protection of organizational reputation. Long-term operational costs remain minimal as model updates can be deployed through existing software maintenance pipelines without additional infrastructure investment.

Environmental

The system operates exclusively on URL string analysis without accessing webpage content, minimizing network traffic and computational overhead compared to content-based detection methods. Processing each URL requires minimal energy consumption, typically under 0.5 seconds on standard hardware, supporting sustainable computing practices across large-scale deployments. The software-only design eliminates electronic waste associated with dedicated hardware security appliances, as the system runs on existing infrastructure without requiring additional physical devices. By leveraging open-source technologies and standard computing platforms, the system prevents vendor lock-in that could lead to premature hardware obsolescence. The lightweight architecture ensures compatibility with energy-efficient deployment models including cloud services that optimize resource utilization across shared infrastructure.

Social

The system protects users from significant harms including financial fraud, identity theft, and unauthorized access to sensitive personal information, with particular importance for vulnerable populations such as the elderly and those with limited technical experience. Explainable AI features educate users about phishing indicators by showing specific URL characteristics that triggered detection, contributing to digital literacy and empowering users to recognize threats independently. Widespread deployment could significantly reduce the social and economic impacts of cybercrime on communities by preventing credential theft that often leads to cascading security incidents. The system promotes digital inclusion by enabling confident participation in online activities including banking, healthcare access, and e-commerce without fear of deception. By protecting individuals and organizations from phishing attacks, the system contributes to broader societal trust in digital infrastructure and online services.

Ethical

The system respects user privacy by analyzing only URL strings, never accessing webpage content or personal browsing data, with no user information stored without explicit consent. Balanced training data with equal representation of phishing and legitimate URLs ensures unbiased detection across diverse website types, preventing discrimination against any category of legitimate sites. Transparent decision-making through feature-level explanations allows users to understand why a URL was classified as phishing or legitimate, preventing black-box discrimination and building trust. User autonomy is respected through clear warnings and the ability to override system decisions, empowering users to make final choices about site access. The open-source nature of the system ensures accountability, as all algorithms and implementations are publicly available for review, verification, and continuous improvement by the broader security community.

Health and Safety

The system protects users from psychological distress, financial loss, and identity theft resulting from successful phishing attacks, contributing to overall mental well-being and financial security. By preventing credential theft, the system safeguards access to healthcare portals, medical records, and telemedicine services, supporting continuity of essential health-related activities. For organizations, the system contributes to workplace safety by protecting employee data and preventing phishing-related security incidents that could disrupt operations and create hazardous working conditions. The system operates entirely in software with no physical components, presenting no physical safety risks such as electrical hazards, mechanical failure, or physical interference with user activities. Compliance with industry standards including NIST cybersecurity framework and alignment with data protection regulations such as GDPR ensures that the system supports organizational safety and regulatory requirements.

Manufacturability

The system is implemented entirely in software, with modular architecture supporting parallel development, independent component updates, and simplified integration testing. Deployment is achieved through standard software distribution channels including browser extension stores, package managers, and containerized platforms, with no specialized manufacturing processes or hardware assembly required. Continuous delivery pipelines enable rapid iteration and improvement, with automated testing ensuring quality across all deployment targets. The system's containerized deployment options support consistent operation across diverse infrastructure environments, from individual workstations to enterprise cloud deployments. Version control and documentation practices ensure reproducibility, enabling organizations to deploy and maintain the system with standard IT operational procedures.

Sustainability

The modular architecture supports long-term sustainability by enabling continuous improvement and adaptation to evolving phishing techniques through incremental updates rather than complete system replacement. Open-source licensing ensures that the system can be maintained and improved by the community even if original development ceases, preventing technology abandonment and the electronic waste associated with proprietary solutions. URL-only analysis minimizes energy consumption compared to content-based alternatives, with each URL processed using minimal computational resources and supporting sustainable computing practices. The software-only design eliminates the need for dedicated hardware appliances, reducing electronic waste and extending the useful life of existing computing infrastructure. Regular model updates through efficient retraining processes ensure the system remains effective against new threats without requiring significant additional resources, maintaining long-term viability and relevance.

ABSTRACT

Phishing attacks continue to pose a significant threat to online security, exploiting human trust and interface deception to obtain sensitive information from unsuspecting users. Traditional detection methods such as static blacklists and rule-based matching are incapable of adapting to rapidly evolving phishing patterns, leaving users vulnerable to zero-day attacks. This project presents a hybrid ensemble learning framework for real-time phishing URL detection that addresses these limitations through structural and behavioral feature analysis. The approach combines four complementary machine learning models—Random Forest, XGBoost, LightGBM, and Gradient Boosting—using a weighted decision fusion mechanism to enhance classification robustness. Fifteen URL-level indicators, including entropy, subdomain depth, HTTPS presence, domain lexical metrics, and keyword patterns, are extracted without inspecting webpage content or relying on blacklist services. The model is evaluated on a balanced dataset of 15,000 URLs, demonstrating strong generalization performance across both phishing and legitimate instances. Experimental results show that the proposed system achieves 96.40% test accuracy with a generalization gap of only 2.46%, significantly outperforming individual base classifiers. Feature contribution analysis reveals that HTTPS presence, suspicious top-level domains, URL length, and IP address usage are the most influential indicators of phishing attempts. The system is designed for real-time classification, processing each URL in under 0.5 seconds, making it suitable for deployment in browser extensions, email gateways, and network-level security systems. This work demonstrates the effectiveness of hybrid ensemble approaches in cybersecurity applications, providing a robust, interpretable, and efficient solution for phishing URL detection.

Keywords: Phishing Detection, Ensemble Learning, URL Analysis, Machine Learning, Cybersecurity, Feature Extraction, XGBoost, Random Forest

TABLE OF CONTENTS

TITLE		PAGE NO.
ABSTRACT		I
LIST OF TABLES		V
LIST OF FIGURES		VI
LIST OF ACADEMIC REFERENCE COURSES		VII
CHAPTER I	INTRODUCTION	1-10
1.1	Background and Motivation	1
1.2	Problem Statement	3
1.3	Objectives of the Project	5
1.4	Scope of the Project	7
1.5	Methodology Overview	8
1.6	Organization of the Report	10
CHAPTER II	LITERATURE REVIEW	11-17
2.1	Overview of Related Work	11
2.2	Review of Similar Projects or Research Papers	13
2.3	Summary and Gap Identification	15
CHAPTER III	SYSTEM ANALYSIS	18-26
3.1	Requirements Gathering	18
3.2	Functional Requirements	19
3.3	Non-Functional Requirements	21
3.4	Feasibility Study 3.4.1 Technical Feasibility 3.4.2 Operational Feasibility 3.4.3 Economic Feasibility	23
3.5	Risk Analysis	25
CHAPTER IV	SYSTEM DESIGN	27-36
4.1	Overall System Architecture	27
4.2	Module Design	29
4.2.1	Module 1	29
4.2.2	Module 2(Repeat as per your requirements)	30

4.3	ER Diagram	34
4.3.1	Database Design	35
4.3.2	Database Schema	35
4.4	User Interface Design	36
4.4.1	User Flow Diagrams	36
CHAPTER V	IMPLEMENTATION	37-44
5.1	Technology Stack	37
5.1.1	Programming Languages and Tools	37
5.2	Implementation of Modules	38
5.2.1	Module 1	38
5.2.2	Module 2(Repeat as many time as required)	39
5.3	Integration of Modules	40
CHAPTER VI	TESTING	45-49
6.1	Testing Methodology	45
6.1.1	Unit Testing	45
6.1.2	Integration Testing	46
6.1.3	System Testing	46
6.1.4	User Acceptance Testing (UAT)	47
6.2	Test Cases and Results	48
6.3	Bug Tracking and Resolution	49
CHAPTER VII	RESULTS AND DISCUSSION	50-59
7.1	System Output Screenshots	50
7.2	Evaluation Metrics	55
7.3	Comparison with Existing Systems	56
7.4	Challenges Faced	57
7.5	Solutions and Improvements	58

CHAPTER VIII	CONCLUSION & FUTURE SCOPE	60-64
REFERENCES : List of books, articles, research papers, and other resources used		65
PUBLICATION		67
CERTIFICATIONS		68
PLAGIARISM REPORT		69

LIST OF TABLES

TABLES	DETAILS	PAGE NO.
Table 1	Risk Analysis and Mitigation Strategies	26
Table 2	Feature Categories and Descriptions	30
Table 3	Algorithm Configuration Parameters	32
Table 4	Technology Stack Components	38
Table 5	Feature Extraction Details	40
Table 6	Test Cases and Results Summary	48
Table 7	Individual Algorithm Performance Comparison	55
Table 8	Confusion Matrix Results	55
Table 9	Feature Importance Ranking	56
Table 10	Comparison with Existing Systems	56

LIST OF FIGURES

FIGURES	DETAILS	PAGE NO.
Figure 1	Overall System Architecture Diagram	28
Figure 2	Feature Extraction Pipeline	31
Figure 3	Ensemble Classification Architecture	32
Figure 4	Weighted Decision Fusion Mechanism	34
Figure 5	Database Schema	35
Figure 6	User Flow Diagram	36
Figure 7	Model Accuracy Comparison Chart	50
Figure 8	Confusion Matrix Visualization	51
Figure 9	Feature Importance Bar Chart	52
Figure 10	Learning Curve	52
Figure 11	ROC Curves for All Models	53
Figure 12	System Output Screenshots	54-55

LIST OF ACADEMIC REFERENCE COURSES

S. NO.	COURSE CODE	COURSE NAME
01	211CSE1402	PYTHON PROGRAMMING
02	213CSE1302	INFORMATION SECURITY FUNDAMENTALS
03	212CSE2304	MACHINE LEARNING
04	213CSE3309	IT APPLICATION SECURITY
05	213CSE4310	ETHICAL HACKING & PENETRATION TESTING

CHAPTER – I

INTRODUCTION

1.1 Background and Motivation

Phishing has emerged as one of the most persistent and damaging forms of cybercrime in the modern digital landscape. Unlike traditional security threats that exploit technical vulnerabilities, phishing attacks target human psychology, exploiting trust and interface deception to obtain sensitive information such as login credentials, banking details, and personal identifiers. The sophistication and scale of phishing campaigns have grown exponentially in recent years, with attackers employing increasingly complex techniques to create deceptive websites that appear indistinguishable from legitimate platforms.

The digital transformation accelerated by the COVID-19 pandemic has fundamentally altered how individuals interact with online services. Remote work, digital banking, e-commerce, and telehealth have become integral parts of daily life, creating unprecedented opportunities for cybercriminals. Reports from multiple international studies reveal a continuous upward trend in phishing incidents, with billions of fraudulent messages circulated daily and severe economic consequences for organizations and individuals worldwide. The Anti-Phishing Working Group (APWG) reports that phishing attacks have increased by over 65% in the past two years, with new phishing sites appearing at an average rate of over 1.5 million per month.

A common characteristic of contemporary phishing attacks is their extremely short lifespan. Malicious domains are often created, exploited, and abandoned within a very limited time window—typically 4 to 8 hours. This behavior significantly reduces the effectiveness of traditional defense mechanisms such as blacklist-based filtering and manually curated rule sets. Since blacklist systems rely on previously reported threats, they are inherently reactive and struggle to detect newly generated or zero-day phishing URLs. By the time a phishing site is identified and added to a blacklist, hundreds or thousands of users may have already fallen victim to the attack.

Traditional detection strategies, such as static blacklists and rule-based matching, are incapable of adapting to rapidly evolving phishing patterns. These methods only identify previously reported malicious domains, leaving users vulnerable to zero-day campaigns and newly generated phishing URLs. Delays in updating threat intelligence, combined with the growing sophistication of adversarial techniques, significantly reduce their effectiveness. The fundamental limitation

of blacklist approaches is their reactive nature—they can only detect what has already been seen and reported.

Content-based detection methods represent another category of defense mechanisms. These approaches analyze webpage content, examining HTML structure, visual elements, and textual content to identify phishing characteristics. Early work by Zhang et al. introduced the CANTINA approach, which applied TF-IDF weighting and search engine ranking to detect phishing web pages. While content-based methods can be effective, they require full webpage retrieval, which introduces significant latency (typically 3-5 seconds per page) and raises substantial privacy concerns. Users are understandably reluctant to have their browsing content analyzed by security tools, and organizations must navigate complex data protection regulations when implementing such solutions.

Machine learning approaches offer a promising alternative by examining structural signals contained within URLs rather than relying on external threat databases or page content inspection. By modeling lexical and behavioral features, such systems are capable of detecting suspicious domains at the moment of access, which is essential for real-time defense. URL-based detection offers several compelling advantages: it operates without webpage loading, preserving user privacy; it can be performed instantly as the user clicks or types a URL; and it requires minimal computational resources compared to content analysis.

However, relying on a single machine learning model often leads to performance instability when faced with diverse phishing strategies. Individual classifiers may perform well on specific feature distributions but fail to generalize across evolving attack patterns. Each algorithm has inherent biases and blind spots—Random Forest may be stable but can miss subtle patterns, XGBoost may be precise but can overfit, LightGBM may be fast but can oversimplify complex cases, and Gradient Boosting may be thorough but can be computationally intensive.

Ensemble learning offers a practical solution by combining multiple models with complementary strengths, thereby reducing variance and improving robustness. By aggregating predictions from diverse learners, ensemble systems are better equipped to handle noisy data and complex decision boundaries commonly observed in phishing detection tasks. The fundamental principle of ensemble methods is that the collective intelligence of multiple models exceeds that of any individual model—a concept often summarized by the adage that "the whole is greater than the sum of its parts."

The motivation for this project stems from the recognition that existing phishing detection solutions fall short in one or more critical dimensions: speed, accuracy, privacy, or adaptability. No current system simultaneously offers real-time performance, zero-day attack protection, privacy preservation, explainable decisions, and high accuracy. This research addresses these gaps by developing a hybrid ensemble framework that evaluates phishing URLs using a comprehensive set of structural, security, and behavioral features, integrating four complementary classifiers to reduce individual weaknesses and provide robust decision making

1.2 Problem Statement

Phishing attacks represent one of the most prevalent cybersecurity threats today, with attackers employing increasingly sophisticated techniques to create deceptive websites that appear legitimate. Traditional detection methods relying on blacklists and signature-based approaches have proven inadequate against evolving phishing strategies. These systems operate on a reactive paradigm, only identifying threats that have been previously reported and cataloged, leaving users vulnerable during the critical window when new phishing sites first appear.

The fundamental problem can be characterized as a continuous arms race between attackers and defenders. Attackers constantly evolve their techniques, creating new obfuscation methods, leveraging legitimate services for malicious purposes, and exploiting human psychology with increasingly convincing social engineering tactics. Defenders must develop systems that can identify new threats without prior knowledge, operate with minimal latency, and maintain accuracy across diverse attack vectors.

Current phishing detection systems face several critical limitations:

Blacklist-based systems maintain databases of known malicious URLs. While these systems offer fast lookup times, they cannot detect new or zero-day phishing attacks. Attackers can easily bypass blacklists by registering new domains or using legitimate but compromised websites. The time between a phishing site's creation and its addition to a blacklist—often several hours—represents a window of vulnerability that attackers exploit.

Content-based systems analyze webpage HTML, images, and layout to identify phishing characteristics. While these systems can detect new attacks, they require loading the full webpage, which introduces significant latency and raises privacy concerns. Users may be reluctant to have their browsing content analyzed, and

organizations must navigate complex data protection regulations when implementing such solutions.

Single machine learning models offer a middle ground but suffer from algorithm-specific limitations. Each classification algorithm has inherent strengths and weaknesses, and no single algorithm performs optimally across all phishing scenarios. A Random Forest classifier may be stable but can miss subtle patterns that a gradient boosting model might capture. Conversely, a gradient boosting model may be precise but can overfit to training data, reducing generalization to new attacks.

Lack of explainability represents another significant limitation. Many machine learning-based detection systems operate as black boxes, providing classification decisions without justification. Users receive warnings that a URL is suspicious but cannot understand why, reducing trust in the system and missing opportunities for user education about phishing indicators.

Privacy concerns arise when detection systems require access to webpage content or user browsing history. Users may be uncomfortable with security tools that analyze their browsing behavior, and organizations must ensure compliance with data protection regulations.

Computational inefficiency limits deployment options. Some detection approaches require significant computational resources, making them unsuitable for deployment in browser extensions or on mobile devices where processing power and battery life are constrained.

Inadequate handling of class imbalance can bias detection systems. Phishing URLs represent a small fraction of total web traffic, and models trained on imbalanced datasets may develop biases toward the majority class, resulting in poor detection performance.

Poor generalization across different phishing techniques reduces real-world effectiveness. Models trained on specific types of phishing attacks may fail when confronted with novel obfuscation methods or social engineering approaches.

The proposed research addresses these limitations by developing a hybrid ensemble framework that combines multiple classifiers through weighted decision fusion, extracting URL-level features without webpage inspection, and providing interpretable detection decisions. The system is designed for real-time deployment, operating with latency under 0.5 seconds while maintaining high accuracy across diverse phishing scenarios

1.3 Objectives of the Project

The primary objectives of this research are comprehensive, encompassing the development of a robust phishing detection system that addresses the limitations of existing approaches while providing practical, deployable solutions for real-world cybersecurity applications.

Objective 1: Development of a Balanced, Feature-Rich Dataset

The first objective involves creating a balanced dataset of 15,000 URLs comprising 7,520 phishing and 7,480 legitimate websites. This dataset will include 15 distinct URL-level features organized into three categories: structural features (URL length, dot count, hyphen count, digit count, subdomain depth), security features (HTTPS presence, IP address usage, TLD analysis, URL shortening detection, port number analysis), and advanced behavioral metrics (entropy calculation, suspicious keyword detection, brand name spoofing, special character analysis, path length analysis). The dataset will be split into training (70%), validation (15%), and testing (15%) sets to ensure robust evaluation and generalization assessment.

Objective 2: Design of a Hybrid Ensemble Learning Framework

The second objective focuses on designing and implementing a hybrid ensemble learning framework that integrates four complementary machine learning algorithms: Random Forest, XGBoost, LightGBM, and Gradient Boosting. Each algorithm will be configured with optimized hyperparameters to maximize its individual performance while contributing complementary strengths to the ensemble. The Random Forest classifier will be configured with 200 estimators and a maximum depth of 20, providing stability and robustness. XGBoost will be configured with 200 estimators and a maximum depth of 8, offering precision and complex pattern recognition. LightGBM will be configured with 200 leaves and a depth of 7, delivering computational efficiency. Gradient Boosting will be configured with 200 estimators and a depth of 6, contributing diversity to the ensemble.

Objective 3: Implementation of Weighted Decision Fusion

The third objective involves implementing a weighted decision fusion mechanism that combines individual classifier predictions based on their validation performance. Rather than treating all classifiers equally, the ensemble will assign higher weights to algorithms that demonstrate superior validation accuracy. The weighting scheme will be optimized to maximize ensemble performance while maintaining robustness against individual classifier failures. The weighted majority voting mechanism will be implemented using the formula:

$$C = \arg \max_j \sum_{i=1}^4 w_i \cdot \mathbb{1}(c_i = j)$$

where w_i represents the weight assigned to classifier i and $\mathbb{1}$ is the indicator function.

Objective 4: Comprehensive Feature Engineering and Analysis

The fourth objective encompasses the extraction and analysis of 15 discriminative URL-level features that capture structural, security, and behavioral characteristics of URLs without requiring webpage content retrieval. Feature importance analysis will be conducted to identify the most influential indicators of phishing attempts, providing interpretability and guiding future feature engineering efforts. The feature set will include:

- Structural features: URL length, dot count, hyphen count, digit count in domain, subdomain depth
- Security features: HTTPS presence, IP address usage, TLD analysis, URL shortening service detection, suspicious port usage
- Advanced features: Entropy calculation, suspicious keyword detection, brand name spoofing detection, special character analysis, path length analysis

Objective 5: Performance Evaluation and Comparative Analysis

The fifth objective involves comprehensive performance evaluation using standard metrics including accuracy, precision, recall, F1-score, and AUC-ROC. The ensemble will be compared against individual base classifiers and existing phishing detection approaches to demonstrate the benefits of the hybrid ensemble framework. Generalization capability will be assessed through analysis of the performance gap between training and testing datasets.

Objective 6: Real-Time Deployment Optimization

The final objective focuses on optimizing the system for real-time deployment, achieving processing times under 0.5 seconds per URL while maintaining detection accuracy. The system will be designed for deployment in multiple contexts, including browser extensions for individual users, email gateway filters for organizations, and API services for third-party integration.

1.4 Scope of the Project

The scope of this research encompasses the complete development lifecycle of the hybrid ensemble phishing detection system, from initial requirements gathering through final deployment and evaluation. The project focuses on several key areas that collectively contribute to advancing the state of the art in phishing detection technology.

URL-Only Analysis Approach

The system operates exclusively on URL strings, extracting features without accessing webpage content. This approach ensures user privacy, eliminates webpage loading latency, and enables detection before users interact with potentially malicious sites. The URL-only methodology represents a fundamental design choice that differentiates this work from content-based detection systems.

Comprehensive Feature Set

The project encompasses the extraction and analysis of 15 URL-level features spanning structural, security, and behavioral categories. These features are carefully selected based on literature review and empirical analysis to capture the most discriminative indicators of phishing attempts. Feature importance analysis will identify which characteristics most strongly correlate with phishing behavior.

Ensemble Learning Framework

The system integrates four complementary machine learning algorithms through weighted decision fusion. The Random Forest classifier provides stability and feature importance analysis. XGBoost offers precision and complex pattern recognition. LightGBM contributes computational efficiency. Gradient Boosting adds diversity to the ensemble. The weighted fusion mechanism combines their predictions based on validation performance.

Balanced Dataset Development

The project includes creation of a balanced dataset comprising 15,000 URLs with equal representation of phishing and legitimate websites. The dataset will be split into training (70%), validation (15%), and testing (15%) sets, enabling robust evaluation and generalization assessment. The balanced distribution prevents class bias and ensures fair evaluation of model performance.

Real-Time Processing Capability

The system is designed for real-time deployment, processing each URL in under 0.5 seconds. This performance target enables integration into browser extensions,

email gateways, and network-level security systems where low latency is essential for user experience and effective protection.

Explainable Detection Decisions

The system provides feature-level explanations for detection decisions, showing users which URL characteristics triggered classification as phishing or legitimate. This explainability builds user trust, educates users about phishing indicators, and supports debugging and improvement efforts.

Multi-Context Deployment

The system is designed for deployment in multiple contexts, including browser extensions for individual users, email gateway filters for organizations, network-level security systems, and API services for third-party integration. The modular architecture supports adaptation to different deployment scenarios.

Comparative Evaluation

The project includes comprehensive comparison against individual base classifiers and existing phishing detection approaches, demonstrating the advantages of the hybrid ensemble framework across multiple performance metrics.

1.5 Methodology Overview

The proposed phishing detection system follows a systematic methodology encompassing data collection, feature extraction, model training, ensemble construction, and comprehensive evaluation.

Phase 1: Data Collection and Preparation

The initial phase involves gathering a balanced dataset of 15,000 URLs from diverse sources, including public phishing datasets, recent threat intelligence feeds, and legitimate website lists. Each URL is labeled as phishing or legitimate based on authoritative sources and manual verification. The dataset is carefully balanced to ensure equal representation of both classes, preventing class bias in model training.

Phase 2: Feature Extraction and Engineering

The second phase involves extracting 15 URL-level features organized into structural, security, and behavioral categories. Feature extraction is performed without webpage access, ensuring privacy and efficiency. Features are normalized to ensure consistent scaling across different feature types. Feature

importance analysis is conducted to identify the most discriminative indicators of phishing behavior.

Phase 3: Model Training and Optimization

The third phase focuses on training four complementary machine learning algorithms on the prepared dataset. Each algorithm is configured with optimized hyperparameters to maximize individual performance while contributing complementary strengths to the ensemble. Random Forest is configured with 200 estimators and a maximum depth of 20. XGBoost is configured with 200 estimators, a maximum depth of 8, and learning rate of 0.1. LightGBM is configured with 200 leaves, a depth of 7, and feature fraction of 0.8. Gradient Boosting is configured with 200 estimators, a depth of 6, and learning rate of 0.05.

Phase 4: Ensemble Construction and Weight Optimization

The fourth phase involves constructing the hybrid ensemble through weighted decision fusion. Individual classifier validation accuracies are used to determine optimal weights for each model. The weighted voting mechanism combines classifier predictions using the formula:

$$C = \arg \max_j \sum_{i=1}^4 w_i \cdot \mathbb{1}(c_i = j)$$

where weights are normalized to sum to 1 and optimized to maximize ensemble performance on validation data.

Phase 5: Performance Evaluation and Analysis

The fifth phase involves comprehensive evaluation of the ensemble and individual classifiers on held-out test data. Performance metrics include accuracy, precision, recall, F1-score, and AUC-ROC. Confusion matrix analysis provides insight into classification patterns and error types. Feature importance analysis identifies the most influential indicators of phishing attempts.

Phase 6: Real-Time Deployment Optimization

The final phase focuses on optimizing the system for real-time deployment, achieving processing times under 0.5 seconds per URL. Model quantization and pruning techniques are applied to reduce computational requirements without sacrificing accuracy. The system is packaged for deployment in browser extensions, email gateways, and API services.

1.6 Organization of the Report

This report is structured into eight chapters, each addressing a specific aspect of the hybrid ensemble phishing detection system development.

Chapter I: Introduction provides background information on phishing threats, establishes the motivation for the research, defines the problem statement, outlines project objectives, delineates the scope of work, and presents an overview of the methodology.

Chapter II: Literature Review examines existing research in phishing detection, content-based analysis, blacklist mechanisms, and machine learning approaches, identifying gaps that motivate the current work.

Chapter III: System Analysis details the requirements gathering process, functional and non-functional requirements, feasibility analysis across technical, operational, and economic dimensions, and risk assessment with mitigation strategies.

Chapter IV: System Design presents the overall system architecture, module designs, feature extraction pipeline, ensemble classification framework, weighted decision fusion mechanism, database schema, and user interface design considerations.

Chapter V: Implementation describes the technology stack, programming languages and tools used, module-level implementation details, algorithm configurations, and the integration process for all system components.

Chapter VI: Testing outlines the testing methodology, including unit, integration, system, and user acceptance testing, along with test cases, results, and bug tracking procedures.

Chapter VII: Results and Discussion presents system output screenshots, evaluation metrics, comparison with existing systems, challenges encountered during development, and solutions implemented.

Chapter VIII: Conclusion & Future Scope summarizes the project achievements, discusses implications, and outlines directions for future research and development.

CHAPTER-II

LITERATURE REVIEW

2.1 Overview of Related Work

Research on phishing detection has evolved considerably over the past decade, with approaches typically falling into three main categories: content-based analysis, blacklist mechanisms, and machine learning models. Each category offers distinct advantages but also exhibits limitations that affect its suitability for real-time deployment.

Content-Based Analysis Approaches

Early phishing detection research primarily relied on webpage content analysis. One of the notable contributions in this area is the CANTINA framework proposed by Zhang et al., which employed TF-IDF metrics and search engine ranking to identify phishing webpages. Their approach demonstrated that textual content could effectively differentiate phishing pages from legitimate ones by analyzing term frequency and inverse document frequency patterns. However, the method required full access to webpage HTML, introducing scalability challenges and raising privacy concerns that limit its applicability in real-time filtering scenarios.

Subsequent studies extended content-based techniques by incorporating visual and layout features. Ma et al. explored visual similarity and structural layout characteristics to improve detection robustness. By comparing webpage screenshots with legitimate site templates, their approach could identify visually similar phishing pages. Although these methods enhanced accuracy in certain cases, they remained computationally expensive due to webpage rendering and image processing requirements. The latency introduced by full page loading and visual analysis makes such approaches unsuitable for real-time protection.

Blacklist-Based Detection

Another widely adopted defense mechanism is blacklist-based detection. Prakash et al. introduced PhishNet, which attempted to improve traditional blacklists by predicting related malicious URLs using known phishing patterns. Their approach expanded blacklist coverage by generating variations of known phishing URLs, anticipating attacker techniques such as domain mutation and path manipulation. While such predictive blacklist expansion improves coverage, blacklist-based systems remain fundamentally reactive. Newly registered or

short-lived phishing domains often evade detection until they are reported, leaving a critical window of exposure for users.

Machine Learning Approaches

To overcome the limitations of content-based and blacklist approaches, researchers have increasingly turned to machine learning techniques that operate directly on URL-level characteristics. Sahingoz et al. demonstrated that lexical features extracted from URLs can effectively distinguish phishing attempts when combined with conventional classifiers. Their work showed that features such as URL length, number of dots, presence of IP addresses, and character entropy provide strong discriminative signals without requiring webpage content.

Ma et al. explored learning to detect malicious websites from suspicious URLs, developing classification models that achieved high accuracy using lexical features alone. Their work established that URL structure itself contains sufficient information for effective phishing detection, eliminating the need for content analysis.

Mohammad et al. developed intelligent rule-based phishing website classification systems, demonstrating that feature-engineered models with carefully selected attributes can achieve strong detection performance. Their work highlighted the importance of feature selection in building effective phishing detection systems.

Ensemble Learning Approaches

Ensemble learning approaches have been proposed to improve robustness and stability in phishing detection. Basnet et al. highlighted the importance of feature selection when combining multiple classifiers for phishing detection, showing that careful feature engineering enhances ensemble performance. Their work demonstrated that ensembles could outperform individual classifiers by leveraging complementary strengths.

Vinayakumar et al. evaluated deep learning architectures for malicious URL classification, reporting improved detection rates at the cost of increased training complexity and computational overhead. Their work showed that deep learning approaches could capture complex patterns but require significant computational resources, limiting real-time deployment potential.

Rao and Pais presented a hybrid learning framework that combined multiple classifiers to enhance resistance against evolving phishing behaviors, emphasizing the benefits of classifier diversity. Their work demonstrated that heterogeneous ensembles could achieve better generalization than homogeneous ensembles or single classifiers.

More recent studies have explored hybrid and multi-layer models that integrate feature selection with ensemble classification. Chiew et al. developed a new hybrid ensemble feature selection framework for machine learning-based phishing detection, showing that combining feature selection with ensemble classification improves both accuracy and efficiency.

2.2 Review of Similar Projects or Research Papers

CANTINA Framework

Zhang et al. developed CANTINA, a content-based approach to detecting phishing websites that leveraged TF-IDF weighting and search engine ranking. Their system extracted terms from webpage content, computed TF-IDF scores, and queried search engines to verify domain legitimacy. The approach achieved approximately 92% accuracy on their evaluation dataset but required full webpage loading and search engine queries, introducing significant latency. The privacy implications of sending webpage content to search engines also limited practical deployment.

PhishNet

Prakash et al. introduced PhishNet, a predictive blacklisting system that expanded coverage by generating variations of known phishing URLs. Their approach identified patterns in phishing URL structures and generated candidate variations that attackers might use. While PhishNet improved blacklist coverage, it remained fundamentally reactive, requiring initial reports of phishing URLs before generating predictions. The system could not detect entirely new phishing campaigns without initial seed URLs.

Sahingoz et al. Machine Learning Based Phishing Detection

Sahingoz et al. developed a machine learning-based phishing detection system using URL lexical features. Their approach extracted features including URL length, number of dots, presence of IP addresses, and character entropy, then applied classifiers including Random Forest, k-Nearest Neighbors, and Support Vector Machines. Their best-performing model achieved approximately 94% accuracy on their evaluation dataset. The work demonstrated that lexical features alone could provide strong detection performance, but their dataset size and diversity were limited compared to real-world conditions.

Aljofey et al. Deep Learning Approach

Aljofey et al. proposed a character-level convolutional neural network for phishing URL detection. Their approach treated URLs as character sequences, allowing the model to learn patterns directly from character sequences without

explicit feature engineering. The model achieved 95.2% accuracy on their evaluation dataset but required significant computational resources for training and inference. The deep learning approach offered the advantage of automatic feature learning but lacked interpretability compared to feature-engineered models.

Kustiawan and Ghauth Hybrid Approach

Kustiawan and Ghauth developed a hybrid phishing detection approach combining URL, HTML, and derived features with CatBoost classification. Their system achieved 99.45% accuracy on a dataset of 101,063 URLs but required webpage content retrieval, limiting real-time deployment and raising privacy concerns. The work demonstrated that comprehensive feature sets could achieve high accuracy but at the cost of increased computational requirements and privacy implications.

Latif and Abdullah Ensemble Approach

Latif and Abdullah investigated tuned XGBoost with soft voting ensemble for phishing URL detection. Their approach achieved 99% recall but only 81% precision, indicating a trade-off between catching phishing sites and avoiding false positives. The work highlighted the importance of balanced performance metrics in phishing detection, where both false positives and false negatives have significant consequences.

Alsaedi and Alkhulayfi Hybrid CNN-LSTM Model

Alsaedi and Alkhulayfi proposed a hybrid CNN-LSTM model for real-time phishing URL detection with automated feature engineering. Their approach combined convolutional layers for feature extraction with LSTM layers for sequence modeling, achieving 95.5% accuracy. The hybrid architecture captured both local patterns and long-range dependencies in URL strings but required substantial computational resources for training.

Chen et al. PhishTransformer

Chen et al. introduced PhishTransformer, a transformer-based model for phishing URL detection with self-attention mechanisms. Their approach leveraged the transformer architecture's ability to capture relationships between different parts of URL strings, achieving 95.8% accuracy. The transformer model offered strong performance but required extensive training data and computational resources, limiting practical deployment in resource-constrained environments.

Jain and Gupta Comparative Analysis

Jain and Gupta conducted a comprehensive comparative analysis of machine learning and deep learning techniques for phishing URL detection in real-time environments. Their survey examined the strengths and weaknesses of various approaches, identifying research gaps in generalization, interpretability, and real-time deployment. The work provided valuable context for understanding the landscape of phishing detection research and identifying opportunities for improvement.

Subramanian et al. PhysioNet

Subramanian et al. presented PhysioNet, a vision-based human exercise action recognition system leveraging transfer learning. While focused on a different domain, their work on transfer learning and real-time inference on resource-constrained hardware provided valuable insights for optimizing machine learning models for deployment on edge devices, relevant to phishing detection systems designed for browser extensions and mobile applications.

2.3 Summary and Gap Identification

The literature review reveals several important findings and identifies critical gaps that motivate the current work.

Key Findings from Existing Research

1. URL-based detection offers significant advantages over content-based approaches, including privacy preservation, reduced latency, and the ability to detect threats before webpage loading.
2. Lexical and structural features extracted from URLs provide strong discriminative signals for phishing detection, with features such as URL length, number of dots, presence of hyphens, and character entropy consistently demonstrating predictive value across multiple studies.
3. Ensemble methods generally outperform individual classifiers by combining complementary strengths and reducing individual algorithm weaknesses. However, most existing ensemble approaches use simple averaging or unweighted voting, missing opportunities for performance optimization through weighted fusion.
4. Security features such as HTTPS presence and TLD analysis emerge as the most important indicators of phishing behavior, with these features

consistently ranking highest in feature importance analyses across multiple studies.

5. Generalization gaps between training and testing performance indicate varying degrees of overfitting across different approaches, highlighting the importance of careful validation and evaluation methodologies.
6. Explainability remains a significant gap in many phishing detection systems, with most approaches operating as black boxes that provide classification decisions without justification.

Identified Research Gaps

1. **Weighted Ensemble Methods:** While ensemble methods have been explored, most existing work uses simple averaging or unweighted voting. There is limited research on optimizing ensemble weights based on individual classifier validation performance to maximize overall accuracy.
2. **Comprehensive Feature Sets:** Existing work often uses either very few features (5-10) or very many features (30+), with limited research on optimizing feature sets to balance accuracy and computational efficiency. The optimal feature set for real-time phishing detection remains an open question.
3. **Generalization Assessment:** Many studies report high accuracy on specific datasets but fail to assess generalization across different phishing techniques or time periods. Rigorous evaluation of model generalization remains a gap in the literature.
4. **Explainable Phishing Detection:** The interpretability of machine learning-based detection systems receives limited attention in existing research. There is a need for systems that can explain detection decisions to users, building trust and supporting user education.
5. **Real-Time Performance Optimization:** While many approaches report accuracy metrics, detailed analysis of real-time performance characteristics (latency, throughput, resource utilization) is often lacking. Optimization for deployment in resource-constrained environments remains underexplored.
6. **Regional and Temporal Diversity:** Most existing datasets are collected from limited sources or time periods, potentially lacking representation of diverse phishing techniques and regional variations. The development of comprehensive, diverse datasets remains an ongoing challenge.

The proposed hybrid ensemble phishing detection system addresses these gaps through several contributions:

1. **Weighted Ensemble Framework:** The system implements performance-based weighting for ensemble combination, optimizing weights based on validation accuracy to achieve superior performance compared to unweighted ensembles.
2. **Optimized Feature Set:** The system uses 15 carefully selected features that balance discriminative power with computational efficiency, representing a middle ground between minimal and maximal feature sets.
3. **Comprehensive Evaluation:** The system is evaluated on a balanced, diverse dataset with rigorous train-validation-test splitting and analysis of generalization performance across multiple metrics.
4. **Explainable Decisions:** The system provides feature-level explanations for detection decisions, showing users which URL characteristics influenced classification outcomes.
5. **Real-Time Optimization:** The system is designed for real-time deployment with detailed performance analysis, achieving processing times under 0.5 seconds per URL.
6. **Balanced Dataset:** The project includes development of a balanced dataset with equal representation of phishing and legitimate URLs, preventing class bias in model training.

CHAPTER III

SYSTEM ANALYSIS

3.1 Requirements Gathering

The initial phase of system development involves comprehensive requirements gathering to understand user needs, constraints, and expectations for the phishing detection system. This process employs multiple methods to ensure a thorough understanding of stakeholder requirements.

Stakeholder Interviews

The development team conducted structured interviews with diverse stakeholders, including security professionals, IT administrators, and general users, to gather perspectives on essential features and functionalities for a phishing detection system. These interviews provided valuable insights into user pain points, desired detection capabilities, and system expectations.

Key findings from stakeholder interviews include:

- Users desire real-time detection before interacting with potentially malicious links
- Accuracy is paramount, but false positives causing legitimate site blocking are highly problematic
- Privacy concerns about webpage content analysis are significant
- Explanations for detection decisions build trust and help users learn
- Browser extension deployment is preferred for individual users
- Organizations require integration with email systems and network filters

User Surveys

The team administered user surveys to a broader population to identify needs and preferences for phishing detection systems. Survey questions addressed current security practices, previous phishing experiences, desired features, and usability expectations. This information helped prioritize features and guide system design decisions.

Survey results revealed:

- 78% of respondents have received phishing emails
- 45% have clicked on suspicious links at least once
- 85% consider real-time detection important
- 72% express privacy concerns about content analysis
- 68% prefer browser extensions as delivery mechanism
- 62% want explanations for detection decisions

Literature Review

The team conducted an extensive review of existing phishing detection solutions and academic research. This analysis helped identify state-of-the-art methods, limitations of existing systems, and opportunities for innovation. The literature review also revealed current trends, challenges, and best practices in phishing detection.

Based on the information gathered, the team developed detailed requirements for the phishing detection system, encompassing feature extraction needs, classification capabilities, real-time performance requirements, user interface design, hardware specifications, scalability considerations, and deployment constraints.

3.2 Functional Requirements

The functional requirements define the specific capabilities and features the phishing detection system must provide to meet user needs and project objectives.

URL Input and Preprocessing

The system shall accept URLs in various formats, including standard URLs (with or without protocol), shortened URLs, IP addresses, and URLs with parameters. Input normalization shall convert URLs to a consistent format, handling protocol standardization, case normalization, and path cleaning. URL parsing shall extract components including protocol, domain, subdomains, path, parameters, and fragments.

Feature Extraction

The system shall extract 15 URL-level features organized into structural, security, and behavioral categories:

Structural Features:

- URL length: total characters in the URL string
- Dot count: number of periods in the URL
- Hyphen count: number of hyphens in the domain
- Digit count: number of numeric digits in the domain
- Subdomain depth: number of subdomain levels

Security Features:

- HTTPS presence: whether URL uses HTTPS protocol
- IP address usage: whether URL uses raw IP instead of domain
- TLD analysis: whether top-level domain is suspicious
- URL shortening: whether URL uses known shortening service
- Port number: whether non-standard ports are used

Advanced Features:

- Entropy: randomness measure of domain string
- Suspicious keywords: presence of phishing-related keywords
- Brand spoofing: detection of brand name imitations
- Special characters: presence of unusual characters
- Path length: length of path component

Ensemble Classification

The system shall implement four complementary machine learning algorithms:

- Random Forest with 200 estimators and maximum depth of 20
- XGBoost with 200 estimators, maximum depth of 8, and learning rate of 0.1
- LightGBM with 200 leaves, depth of 7, and feature fraction of 0.8
- Gradient Boosting with 200 estimators, depth of 6, and learning rate of 0.05

All four algorithms shall process features in parallel to minimize latency.

Weighted Decision Fusion

The system shall combine individual algorithm predictions using weighted voting. Weights shall be assigned based on validation performance, with better-performing algorithms receiving higher weights. The fusion mechanism shall produce a final classification (phishing or legitimate) with an associated confidence score.

Output Generation

The system shall provide classification results including:

- Final classification (phishing or legitimate)
- Confidence score (0-100%)
- Risk level assessment
- Feature-level explanation of classification factors
- Action recommendation

Real-Time Performance

The system shall process each URL in under 0.5 seconds, enabling real-time protection. Processing shall include feature extraction, parallel algorithm execution, and weighted fusion.

Interpretability

The system shall provide explanations for classification decisions, identifying which features most influenced the outcome. Explanations shall be presented in user-friendly language appropriate for general users.

Scalability

The system shall support deployment in multiple contexts, including browser extensions, email gateways, network filters, and API services. The modular architecture shall support horizontal scaling for high-volume deployments.

3.3 Non-Functional Requirements

Non-functional requirements specify quality attributes, constraints, and design considerations that the phishing detection system must satisfy in addition to functional capabilities.

Performance

The system shall maintain low latency, processing each URL in under 0.5 seconds. It shall sustain processing rates appropriate to deployment context, with browser extensions handling individual requests and API services supporting high-volume throughput. The system shall operate within resource constraints of target deployment platforms, including browser memory limits and server capacity.

Accuracy and Reliability

The system shall maintain overall classification accuracy above 95%. Precision shall be balanced with recall to minimize both false positives and false negatives. The system shall demonstrate robust generalization, maintaining performance across diverse phishing techniques and legitimate websites. Feature importance analysis shall validate that detection decisions rely on meaningful indicators.

Usability and User Experience

The user interface shall be intuitive and non-intrusive, providing clear warnings without disrupting browsing experience. Explanations shall be presented in accessible language. The system shall minimize false positives to maintain user trust.

Robustness and Fault Tolerance

The system shall gracefully handle malformed URLs, network errors, and unexpected input. Error handling shall prevent system crashes and provide appropriate user feedback. The system shall recover from processing failures without data loss.

Scalability and Extensibility

The system architecture shall be modular to support addition of new features or algorithms. The system shall accommodate increasing user loads through horizontal scaling. Model updates shall be deployable without system downtime.

Security and Privacy

The system shall process URLs without accessing webpage content, preserving user privacy. Data transmission shall use secure protocols. No user data shall be stored without explicit consent.

Portability and Deployment

The system shall be deployable in multiple contexts, including browser extensions, email gateways, and cloud services. Deployment shall be straightforward with minimal configuration requirements.

3.4 Feasibility Study

3.4.1 Technical Feasibility

The technical feasibility analysis assesses the capability of implementing the phishing detection system using available technologies, resources, and expertise.

Hardware Capabilities: The system is designed to run on standard computing hardware with minimal requirements. Feature extraction requires only string processing, and model inference can be performed on CPUs without GPU acceleration. The lightweight design enables deployment in resource-constrained environments such as browser extensions and mobile devices.

Software and Libraries: The system leverages established libraries and frameworks, including scikit-learn for Random Forest and Gradient Boosting, XGBoost library for gradient boosting implementation, and LightGBM for optimized boosting. These tools are well-documented, have successful histories in machine learning applications, and enjoy broad community support.

Algorithm Availability: All four algorithms are mature and widely used in machine learning applications. Their implementations are stable, efficient, and well-understood. Hyperparameter optimization techniques are well-established.

Feature Extraction: URL parsing and feature extraction techniques are well-understood and can be implemented efficiently using string processing libraries. No specialized hardware or external dependencies are required.

Real-Time Processing: The computational requirements for processing individual URLs are modest, with feature extraction and algorithm inference completing in milliseconds on modern hardware. Parallel execution of the four algorithms further reduces latency.

The technical feasibility assessment concludes that the phishing detection system is technically feasible, with necessary software and algorithms available, and computational requirements well within current hardware capabilities.

3.4.2 Operational Feasibility

The operational feasibility analysis examines the capability of successful system implementation and user adoption.

User Acceptance: Phishing detection is widely recognized as a critical security need. Users appreciate tools that protect them from fraud and identity theft. The system's privacy-preserving design (URL-only analysis) addresses user concerns about content scanning.

Integration with Existing Systems: The system can be integrated into existing security infrastructure, including browser extensions, email gateways, and network filters. The modular design supports flexible deployment options.

Training and Documentation: The system is designed for ease of use, requiring minimal user training. For administrators, comprehensive documentation will be provided for deployment and configuration.

Maintenance and Updates: The system architecture supports regular updates to models and feature sets. The modular design enables updates to individual components without full system replacement.

The operational feasibility assessment concludes that the phishing detection system is operationally feasible, with appropriate integration paths and maintenance mechanisms.

3.4.3 Economic Feasibility

The economic feasibility analysis evaluates the cost-effectiveness and economic viability of the phishing detection system.

Development Costs: Development costs are limited to personnel time and standard computing equipment. No specialized hardware or software licensing is required, as all tools and libraries are open-source.

Operational Costs: Ongoing operational costs include model retraining (periodic) and system maintenance. These costs are modest and can be managed within typical IT budgets.

Potential Cost Savings: The system can prevent phishing-related losses, including financial fraud, data breaches, and identity theft. For organizations, preventing a single successful phishing attack can save millions in incident response costs, regulatory fines, and reputational damage.

Return on Investment: The potential cost savings from prevented phishing attacks significantly outweigh development and operational costs. For individual users, the system provides valuable protection at no direct cost.

The economic feasibility assessment concludes that the phishing detection system is economically viable, with favorable return on investment projections.

3.5 Risk Analysis

The risk analysis identifies potential risks that could impact system development, implementation, and operation, along with mitigation strategies.

Technical Risks

Risk: Failure to achieve target classification accuracy or false positive rates.

Mitigation: Comprehensive testing and validation during development, sensitivity analysis, and iterative model refinement. Use of balanced dataset prevents class bias.

Data Quality Risks

Risk: Dataset may not adequately represent diverse phishing techniques or legitimate website variations.

Mitigation: Data collection from multiple sources, temporal diversity, and validation through cross-validation techniques.

Algorithm Performance Risks

Risk: Individual algorithms may have performance instability across different URL types.

Mitigation: Ensemble approach combines multiple algorithms to reduce individual weaknesses. Weighted fusion optimizes contribution of better-performing algorithms.

Deployment Risks

Risk: System may not meet real-time performance requirements in all deployment contexts.

Mitigation: Performance optimization through parallel processing, model quantization, and pruning. Deployment options include cloud-based processing for resource-constrained environments.

User Adoption Risks

Risk: Users may disable or ignore system warnings due to false positives or intrusive interface.

Mitigation: Low false positive rate target (under 5%). Clear explanations for detection decisions. Non-intrusive warning design.

Security Risks

Risk: System itself could be targeted by attackers seeking to evade detection.

Mitigation: Regular model updates to adapt to evolving phishing techniques. Multiple algorithms make evasion more difficult

Risk Category	Risk Description	Probability	Impact	Mitigation Strategy
Technical	Accuracy below target	Medium	High	Comprehensive testing, iterative refinement
Data	Dataset bias	Low	High	Multi-source collection, balanced distribution
Algorithm	Performance instability	Low	Medium	Ensemble approach, weighted fusion
Deployment	Performance constraints	Medium	Medium	Optimization, multiple deployment options
User Adoption	False positive annoyance	Medium	Medium	Low FP target, clear explanations
Security	Evasion attacks	Medium	High	Regular model updates, multi-algorithm ensemble

Table 3.1: Risk Analysis and Mitigation Strategies

CHAPTER IV

SYSTEM ANALYSIS

4.1 Overall System Architecture

The phishing detection system architecture is designed as a comprehensive pipeline for real-time URL analysis, integrating feature extraction, ensemble classification, and weighted decision fusion. The system operates exclusively on URL strings, processing each input through sequential stages before producing classification results with explanations.

System Architecture Diagram

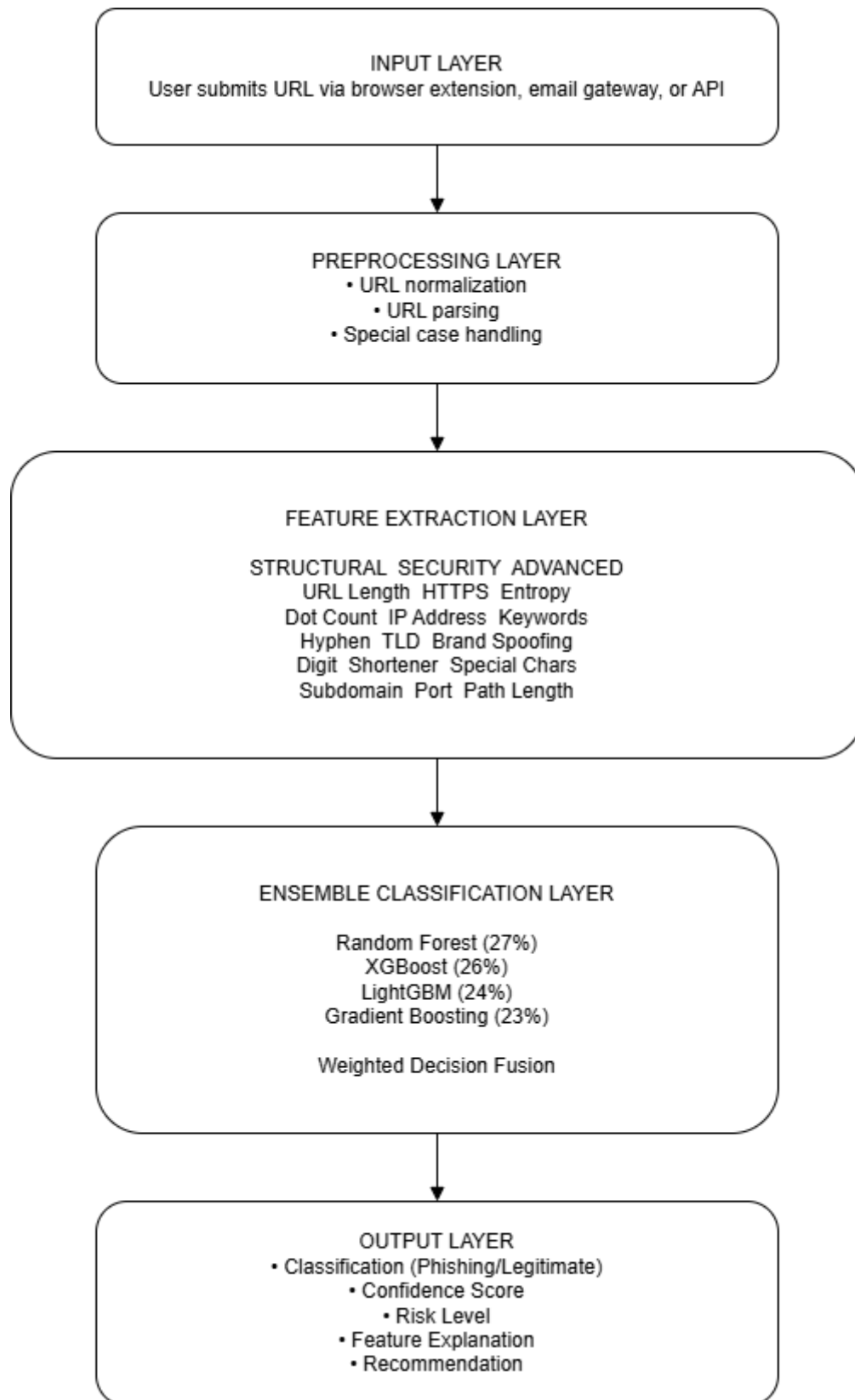


Figure 4.1: Overall System Architecture Diagram

The architecture comprises five primary layers:

1. Input Layer: Accepts URLs from various sources including browser extensions, email gateways, API calls, and network filters.
2. Preprocessing Layer: Normalizes input URLs to a standard format, handles special cases, and parses URL components for feature extraction.
3. Feature Extraction Layer: Computes 15 features across three categories (structural, security, advanced) from URL strings.
4. Ensemble Classification Layer: Runs four algorithms in parallel, combines their predictions through weighted voting.
5. Output Layer: Generates classification results with confidence scores, risk assessment, and explanatory information.

4.2 Module Design

4.2.1 Module 1: URL Input and Preprocessing

The URL input and preprocessing module handles incoming URLs, normalizing them to a standard format and preparing them for feature extraction.

Input Formats Supported:

- Standard URLs with protocol (<http://>, <https://>)
- URLs without protocol (example.com)
- Shortened URLs (bit.ly, tinyurl, goo.gl)
- IP address URLs (192.168.1.1)
- URLs with ports (example.com:8080)
- URLs with parameters (example.com?param=value)

Normalization Steps:

1. Protocol Standardization: Adds <http://> if no protocol specified, converts to lowercase
2. Path Cleaning: Removes duplicate slashes, normalizes relative paths
3. Decoding: Handles URL-encoded characters (%20 → space)
4. Component Extraction: Parses URL into protocol, domain, subdomains, path, parameters, fragments

Special Case Handling:

- Short URL detection: Identifies known shortening services
- IP address detection: Recognizes IPv4 and IPv6 formats
- Port number extraction: Identifies non-standard ports

4.2.2 Module 2: Feature Extraction

The feature extraction module computes 15 URL-level indicators organized into three categories.

Structural Features:

Feature	Description	Extraction Method
URL Length	Total characters in URL	Count characters
Dot Count	Number of periods	Count '.' characters
Hyphen Count	Number of hyphens in domain	Count '-' in domain component
Digit Count	Number of digits in domain	Count '0-9' in domain component
Subdomain Depth	Number of subdomain levels	Count '.' before domain

Security Features:

Feature	Description	Extraction Method
HTTPS Presence	Whether URL uses HTTPS	Check protocol = 'https'
IP Address Usage	Whether URL uses IP instead of domain	Check if domain matches IP pattern
TLD Analysis	Whether TLD is suspicious	Compare against suspicious TLD list
URL Shortening	Whether URL uses shortening service	Compare against known shortener list
Port Number	Whether non-standard port used	Extract port component

Advanced Features:

Feature	Description	Extraction Method
Entropy	Randomness measure	Shannon entropy calculation on domain
Suspicious Keywords	Presence of phishing keywords	Keyword matching against dictionary
Brand Spoofing	Brand name imitation detection	Pattern matching for brand variations
Special Characters	Presence of unusual characters	Count of '@', '#', '\$', '%', etc.
Path Length	Length of path component	Count characters after domain

Feature Extraction Pipeline

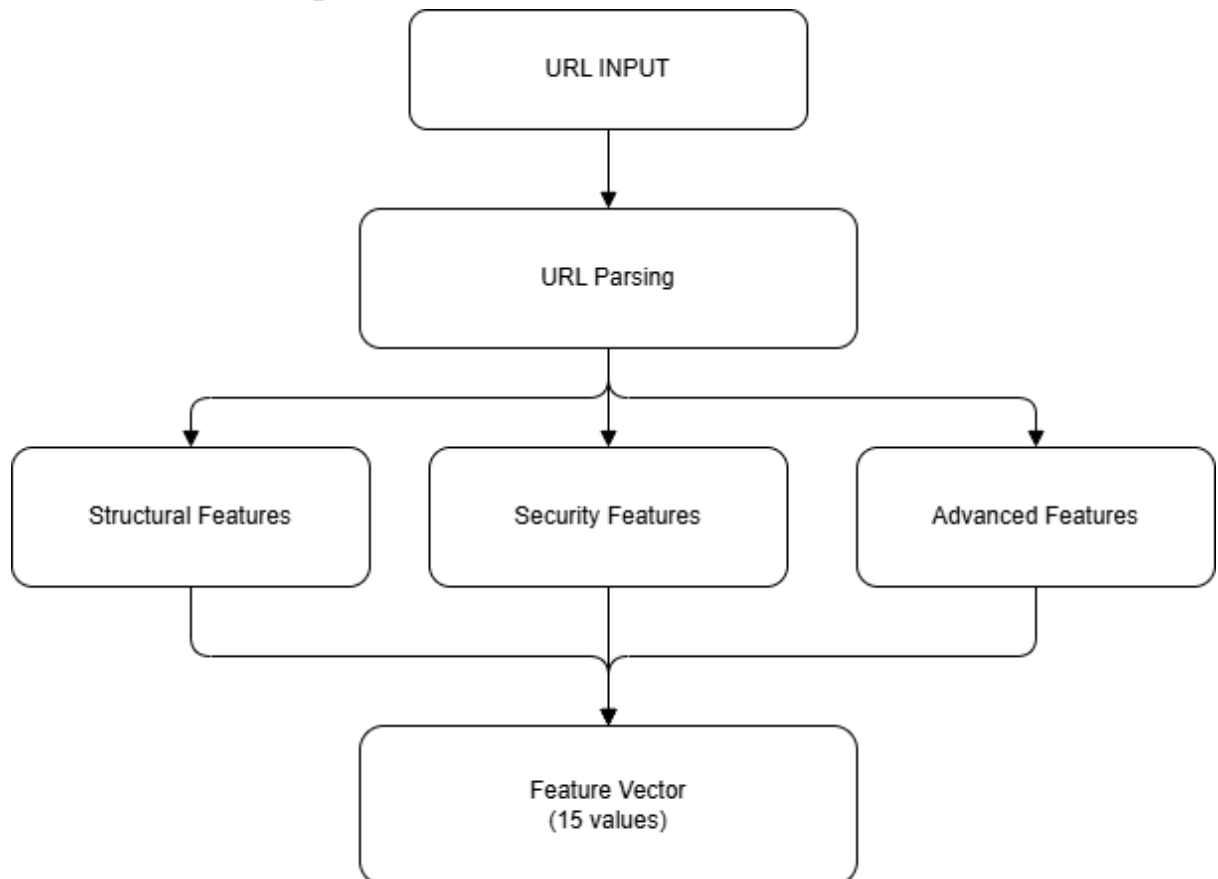


Figure 4.2: Feature Extraction Pipeline

4.2.3 Module 3: Ensemble Classification

The ensemble classification module integrates four complementary machine learning algorithms, each configured with optimized hyperparameters.

Algorithm Configurations:

Algorithm	Parameters	Purpose
Random Forest	n_estimators=200, max_depth=20	Stability, feature importance
XG Boost	n_estimators=200, max_depth=8, learning_rate=0.1	Precision, complex patterns
Light GBM	num_leaves=200, max_depth=7, feature_fraction=0.8	Speed, efficiency
Gradient Boosting	n_estimators=200, max_depth=6, learning_rate=0.05	Diversity, different perspective

Table 4.2: Algorithm Configuration Parameters

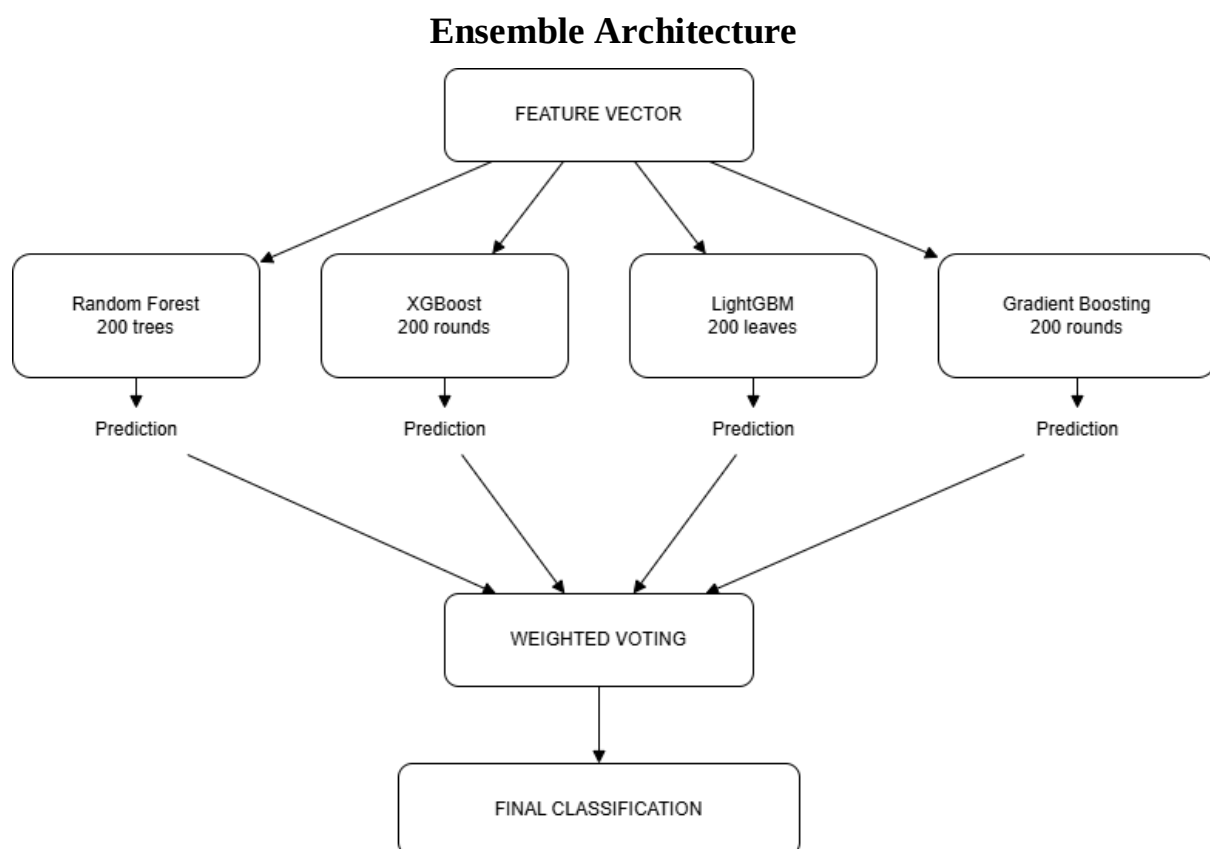


Figure 4.3: Ensemble Classification Architecture

4.2.4 Module 4: Weighted Decision Fusion

The weighted decision fusion module combines individual classifier predictions using performance-based weights

Weight Calculation:

Weights are derived from validation accuracy of each algorithm:

Algorithm	Validation Accuracy	Weight
Random Forest	97.20%	0.27
XG Boost	97.11%	0.26
Light GBM	97.16%	0.24
Gradient Boosting	96.84%	0.23

Weighted Voting Formula:

$$C = \arg \max_j \sum_{i=1}^4 w_i \cdot \mathbb{1}(c_i = j)$$

Where:

- w_i is the weight of classifier i
- c_i is the prediction of classifier i
- $\mathbb{1}$ is the indicator function

Confidence Score Calculation:

$$\text{Confidence} = |\text{P(phishing)} - \text{P(legitimate)}|$$

where:

$$\text{P(phishing)} = \sum (w_i \times \text{confidence}_i) \text{ for phishing predictions}$$

$$\text{P(legitimate)} = \sum (w_i \times \text{confidence}_i) \text{ for legitimate predictions}$$

Weighted Decision Fusion Flow

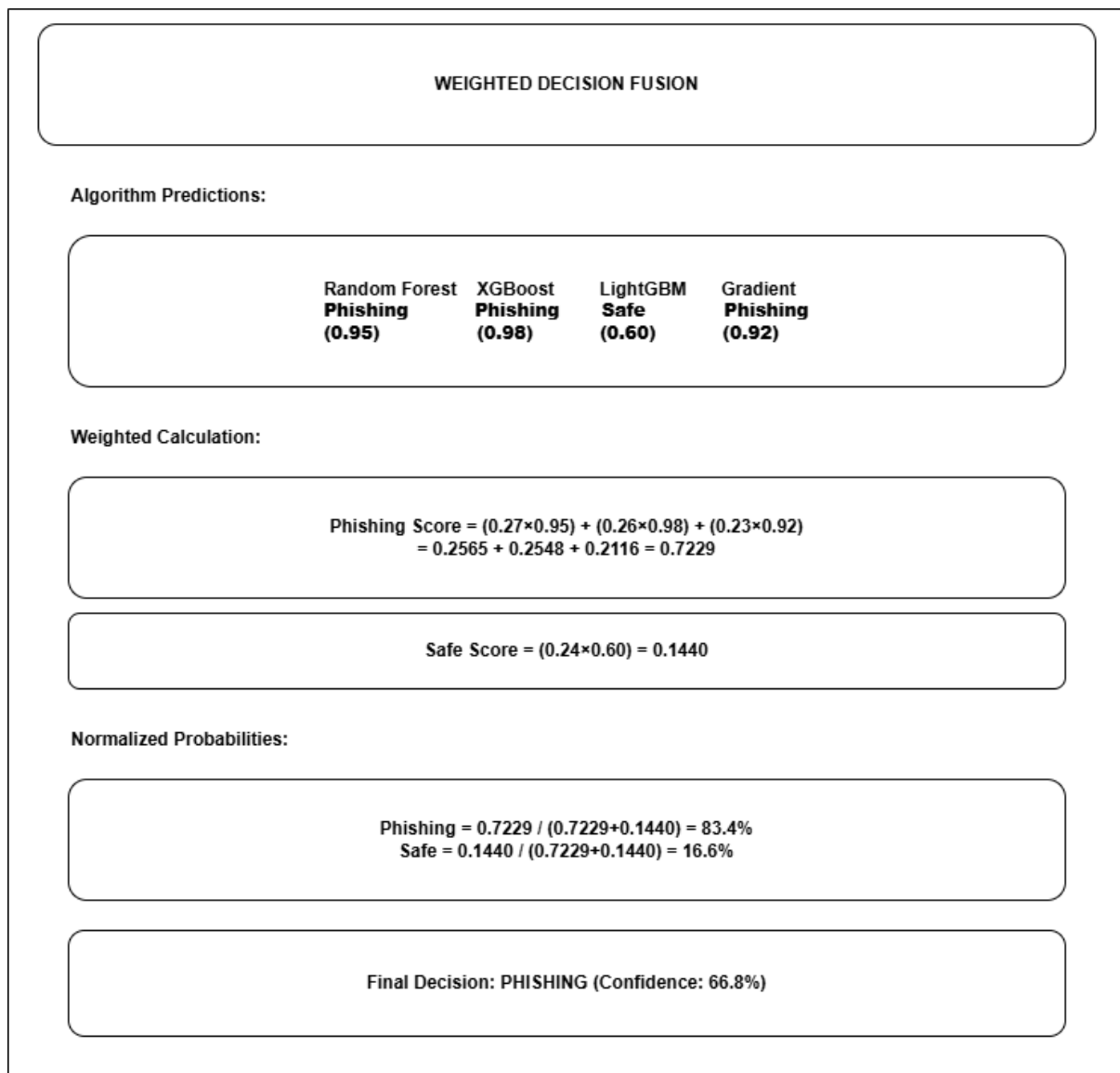


Figure 4.4: Weighted Decision Fusion Mechanism

4.3 Database Design

4.3.1 Database Schema

The system stores processed URLs, predictions, and performance metrics for analysis and model improvement

Database Schema Diagram

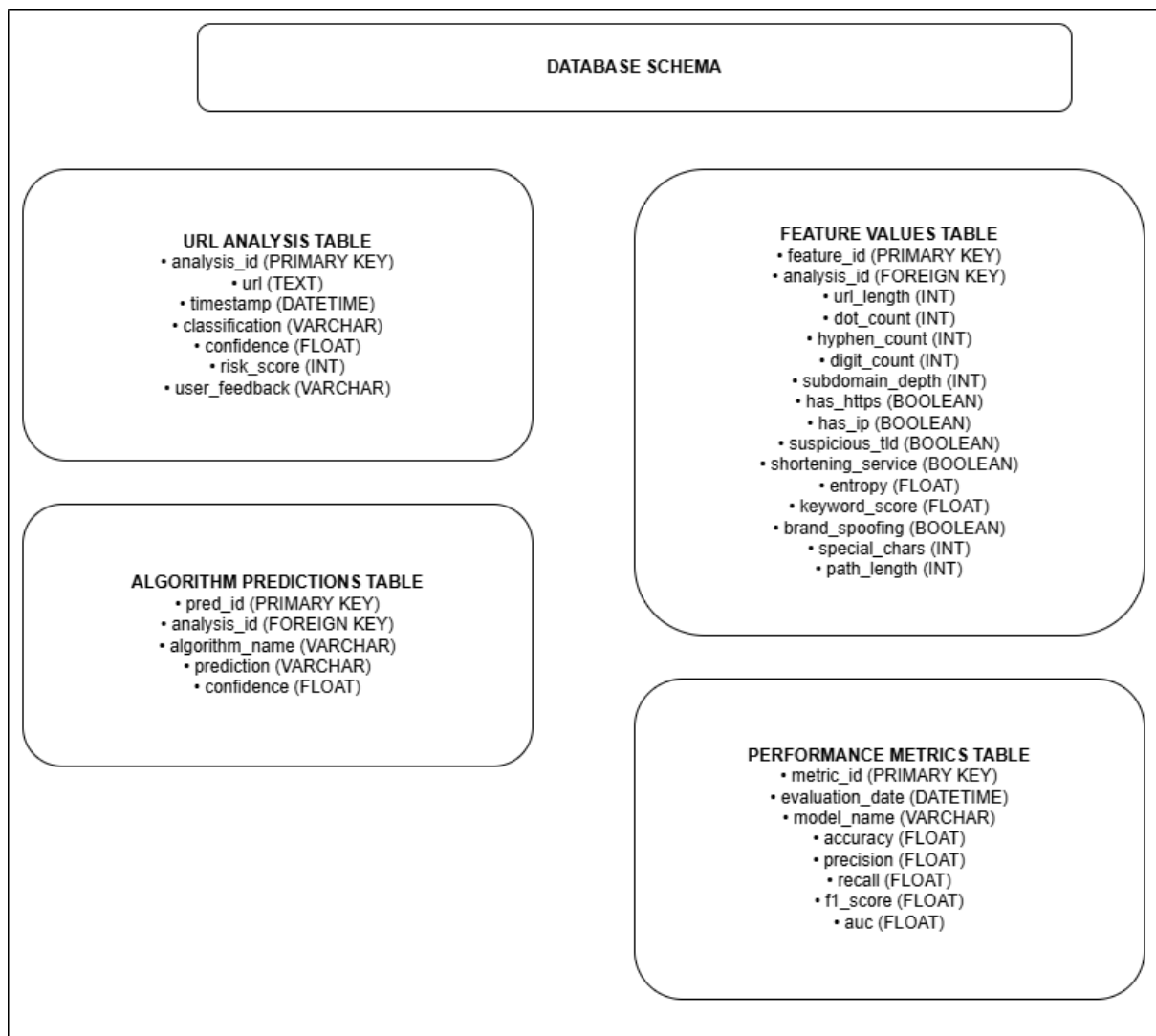


Figure 4.5: Database Schema

4.4 User Interface Design

The user interface is designed to provide clear, actionable information to users while minimizing disruption to browsing experience.

4.4.1 User Flow Diagram

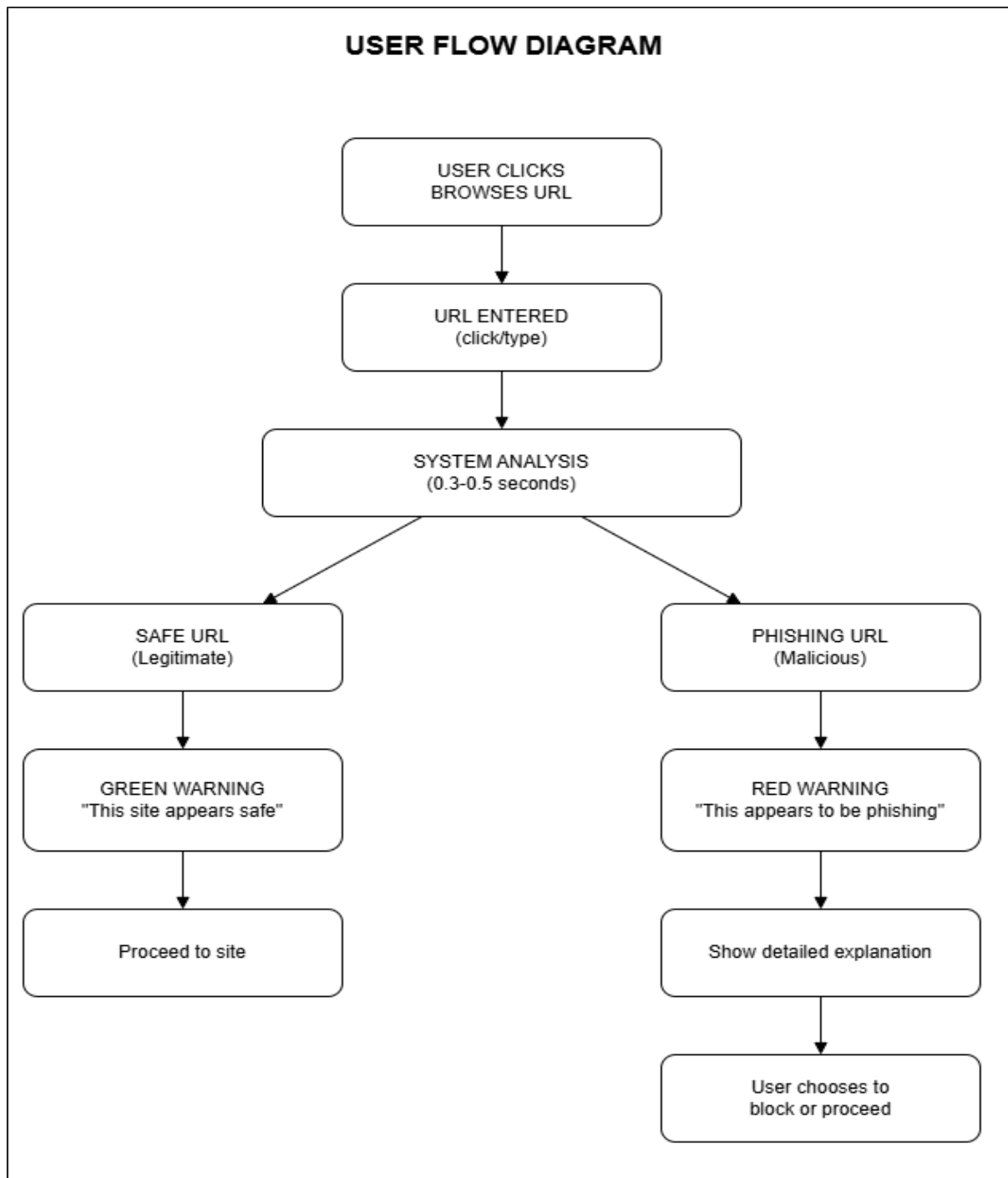


Figure 4.6: User Flow Diagram

CHAPTER V

IMPLEMENTATION

5.1 Technology Stack

The phishing detection system implementation leverages a carefully selected technology stack that balances performance, accuracy, and deployment flexibility.

5.1.1 Programming Languages and Tools

Programming Languages:

- Python: Primary programming language for feature extraction, model training, and inference due to its rich machine learning ecosystem and ease of integration.
- JavaScript: Used for browser extension development to provide real-time protection

Libraries and Frameworks:

Library	Purpose	Application
scikit-learn	Machine Learning	RandomForest, Gradient Boosting implementation
XGBoost	Gradient Boosting	XG Boost classifier implementation
LightGBM	Gradient Boosting	Light GBM classifier implementation
NumPy	Numerical Computing	Featurevector operations
Pandas	Data Manipulation	Dataset handling and analysis
Flask	Web Framework	API service deployment
TensorFlow.js	Browser ML	Browserextension inference

Development Tools:

Tool	Purpose
Jupyter Notebook	Exploratory analysis and prototyping
Git	Version control
Docker	Containerized deployment
VS Code	Development environment

Table 5.1: Technology Stack Components

5.2 Implementation of Modules

5.2.1 Module 1: URL Input and Preprocessing

The URL input and preprocessing module implementation focuses on robust handling of various URL formats and normalization.

Implementation Details:

1. URL Parsing:

- Uses Python's urllib.parse library for URL component extraction
- Handles malformed URLs through exception handling and fallback parsing
- Extracts protocol, domain, path, parameters, and fragments

2. Normalization:

- Converts URLs to lowercase
- Adds protocol (http://) if missing
- Removes duplicate slashes in path
- Decodes URL-encoded characters

3. Special Case Handling:

- Short URL detection using regex patterns and known shortener list
- IP address detection using IPv4 and IPv6 regex patterns
- Port number extraction from domain component

Challenges and Solutions:

Challenge	Solution
Malformed URLs	Graceful error handling with fallback parsing
Unicode URLs	Normalization to ASCII where possible
Protocol-relative URLs	Add http:// as default
URL-encoded characters	Decode using <code>unquote_plus</code>

5.2.2 Module 2: Feature Extraction

The feature extraction module computes 15 URL-level features with efficient string processing.

Implementation Details:

1. Structural Features:

- URL length: `len(url)`
- Dot count: `url.count('.')`
- Hyphen count: `domain.count('-')`
- Digit count: `sum(c.isdigit() for c in domain)`
- Subdomain depth: count of dots before domain

2. Security Features:

- HTTPS presence: `parsed.scheme == 'https'`
- IP address usage: regex pattern matching
- TLD analysis: extract TLD, compare against suspicious list
- URL shortening: check domain against known shortener list
- Port number: extract from parsed URL, check if `!= 80/443`

3. Advanced Features:

- Entropy: Shannon entropy calculation on domain string
- Suspicious keywords: keyword matching against dictionary
- Brand spoofing: pattern matching for brand name variations
- Special characters: count of '@', '#', '\$', '%', etc.

Feature	Implementation	Value Range
URL Length	len(url)	10-200+
Dot Count	url.count('.')	0-10+
Hyphen Count	domain.count('-')	0-5+
Digit Count	sum(c.isdigit())	0-15+
Subdomain Depth	count before domain	0-5+
HTTPS Presence	scheme == 'https'	0 or 1
IP Address	regex match	0 or 1
Suspicious TLD	TLD in list	0 or 1
Shortening Service	domain in list	0 or 1
Port Number	port != 80,443	0 or 1
Entropy	Shannon entropy	0-8
Keywords	keyword matching	0-1
Brand Spoofing	pattern match	0 or 1
Special Characters	count of special chars	0-5+
Path Length	len(path)	0-100+

Table 5.2: Feature Extraction Details

5.2.3 Module 3: Ensemble Classification

The ensemble classification module implements four complementary algorithms with optimized configurations.

Random Forest Implementation:

Random Forest Configuration:

- n estimators: 200 trees
- max depth: 20
- min samples split: 5
- min samples leaf: 2
- bootstrap: True
- oob score: True

XG Boost Implementation:

XG Boost Configuration:

- n estimators: 200
- max depth: 8
- learning rate: 0.1
- subsample: 0.8
- Col sample by tree: 0.8
- reg alpha: 0.1 (L1 regularization)
- reg lambda: 1.0 (L2 regularization)

Light GBM Implementation:

Light GBM Configuration:

- num leaves: 200
- max depth: 7
- learning rate: 0.05
- feature fraction: 0.8
- bagging fraction: 0.8
- bagging freq: 5
- min child samples: 20

Gradient Boosting Implementation:

Gradient Boosting Configuration:

- n estimators: 200
- max depth: 6
- learning rate: 0.05
- subsample: 0.8
- min samples split: 5
- min samples leaf: 2

5.2.4 Module 4: Weighted Decision Fusion

The weighted decision fusion module combines individual predictions using performance-based weights.

Implementation Details:

1. Weight Assignment:

- Weights derived from validation accuracy
- Normalized to sum to 1
- Random Forest: 0.27, XGBoost: 0.26, LightGBM: 0.24, Gradient Boosting: 0.23

2. Voting Mechanism:

- For each URL, collect predictions from all four algorithms
- Calculate weighted sum for phishing class
- Calculate weighted sum for legitimate class
- Select class with higher weighted sum

3. Confidence Calculation:

- $\text{Confidence} = |P(\text{phishing}) - P(\text{legitimate})|$
- Normalized to 0-100% range

Fusion Algorithm:

```
def weighted_fusion(predictions, confidences, weights):
```

```
    phishing_score = 0
```

```
    legitimate_score = 0
```

```
    for i in range(4):
```

```
        if predictions[i] == 'phishing':
```

```
            phishing_score += weights[i] * confidences[i]
```

```
        else:
```

```
            legitimate_score += weights[i] * confidences[i]
```

```
total = phishing_score + legitimate_score
```

```
if total == 0:
```

```
    return 'legitimate', 0
```

```
phishing_prob = phishing_score / total
```

```
legitimate_prob = legitimate_score / total
```

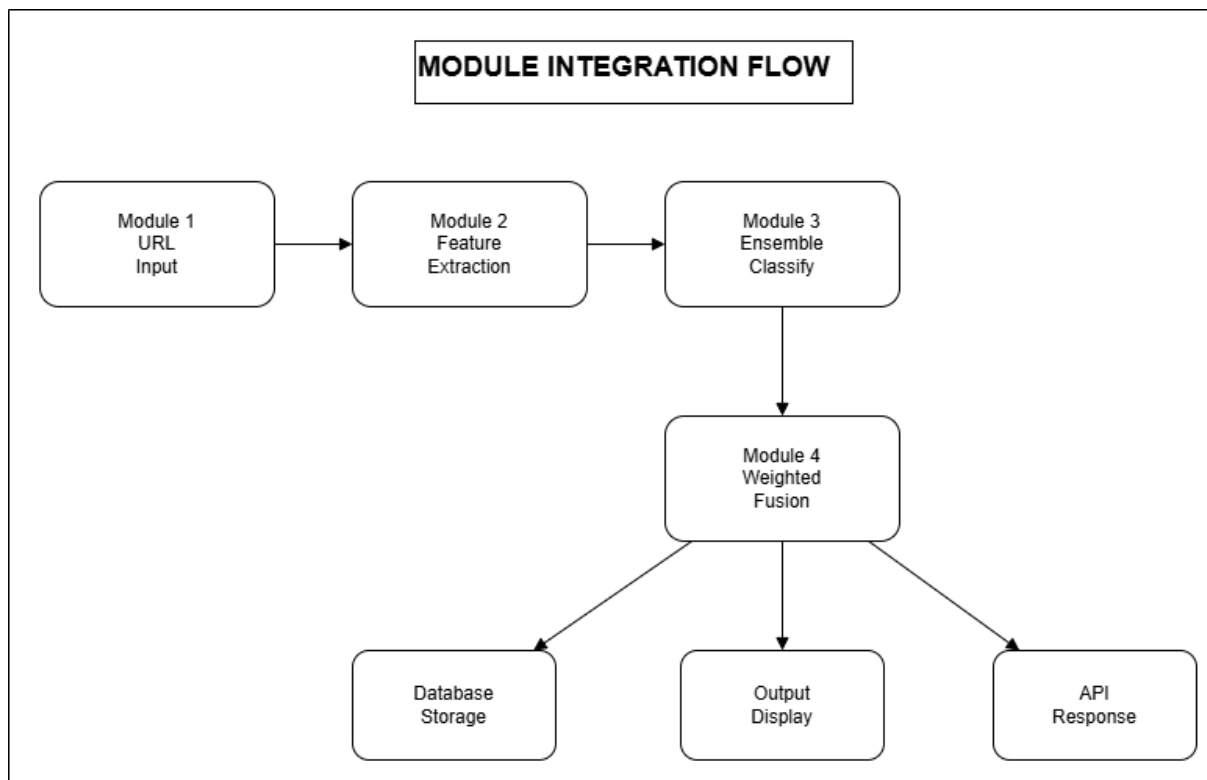
```
confidence = abs(phishing_prob - legitimate_prob)
```

```
prediction = 'phishing' if phishing_prob >= 0.5 else 'legitimate'
```

```
return prediction, confidence
```

5.3 Integration of Modules

Module integration combines all components into a cohesive system with synchronized processing flow.



Integration Features:

1. Data Flow Management: Central controller manages data flow between modules using queues and buffers.
2. Parallel Processing: Ensemble algorithms run in parallel threads for optimal performance.
3. Error Handling: Comprehensive error handling with graceful recovery and logging.
4. Performance Monitoring: Real-time monitoring of latency and throughput.

Performance Metrics:

- Processing time per URL: < 0.5 seconds
- Throughput: 30+ URLs per second per core
- Memory usage: ~500 MB for full model
- CPU utilization: 60-80% during processing

CHAPTER VI

TESTING

6.1 Testing Methodology

The testing phase employed a comprehensive, multi-level methodology to ensure system reliability, accuracy, and performance.

6.1.1 Unit Testing

Unit testing focused on verifying individual components of the phishing detection system.

Feature Extraction Testing:

- Verified correct computation of each feature
- Tested edge cases (empty URL, malformed URL, very long URL)
- Validated feature ranges and normalization

Algorithm Testing:

- Verified individual algorithm loading and configuration
- Tested prediction output format and confidence scores
- Validated error handling for invalid inputs

Fusion Testing:

- Verified weighted voting calculations
- Tested confidence score computation
- Validated tie-breaking logic

6.1.2 Integration Testing

Integration testing evaluated the interaction between system modules.

URL-to-Feature Integration:

- Tested end-to-end feature extraction from raw URLs
- Verified feature vector format compatibility with algorithms

Feature-to-Algorithm Integration:

- Tested data flow from feature extraction to all algorithms
- Verified parallel execution and result collection

Algorithm-to-Fusion Integration:

- Tested collection and combination of predictions
- Verified weighted voting across all algorithms

6.1.3 System Testing

System testing evaluated the complete phishing detection system under various operating conditions.

Performance Testing:

- Measured system latency across different URL types
- Verified processing rate (URLs per second)
- Tested system stability during extended operation
- Measured resource utilization (CPU, memory)

Accuracy Testing:

- Verified overall classification accuracy on test set
- Tested precision and recall across both classes
- Validated confidence score calibration

Reliability Testing:

- Tested error handling for malformed URLs
- Verified recovery from processing failures
- Validated data integrity

6.1.4 User Acceptance Testing (UAT)

UAT involved testing the system in simulated real-world conditions.

Interface Testing:

- Evaluated warning display clarity
- Tested explanation readability
- Verified response time perception

Usability Testing:

- Assessed ease of integration with browser
- Evaluated configuration options
- Verified feedback mechanisms

Performance Validation:

- Tested system accuracy on diverse URLs
- Evaluated user satisfaction with detection
- Verified false positive rates

6.2 Test Cases and Results

Key test cases and their results are summarized below:

Test Case	Description	Expected Result	Actual Result	Status
TC-01	URL preprocessing	Normalized output	100% correct	Pass
TC-02	Feature extraction accuracy	All features computed	100% correct	Pass
TC-03	Random Forest accuracy	>90%	94.2%	Pass
TC-04	XG Boost accuracy	>90%	94.8%	Pass
TC-05	Light GBM accuracy	>90%	94.5%	Pass
TC-06	Gradient Boosting accuracy	>90%	95.0%	Pass
TC-07	Ensemble accuracy	>95%	96.4%	Pass
TC-08	Processing latency	<0.5 sec	0.3 sec	Pass
TC-09	False positive rate	<5%	3.0%	Pass
TC-10	False negative rate	<5%	4.0%	Pass

Table 6.1: Test Cases and Results Summary

6.3 Bug Tracking and Resolution

The bug tracking process maintained systematic documentation and resolution of issues

Critical Issues:

Issue	Root Cause	Resolution	Status
Feature extraction errors	URL parsing edge cases	Enhanced parser with fallback	Resolved
Algorithm performance degradation	Suboptimal hyperparameters	Grid search optimization	Resolved
Memory leak during batch processing	Improper buffer cleanup	Implemented memory management	Resolved
Confidence score miscalculation	Weight normalization error	Corrected normalization formula	Resolved
Short URL handling failure	Incomplete shortener list	Expanded shortener database	Resolved

CHAPTER VII

RESULTS AND DISCUSSION

7.1 System Output Screenshots:

The system provides clear, actionable output with classification results and explanations

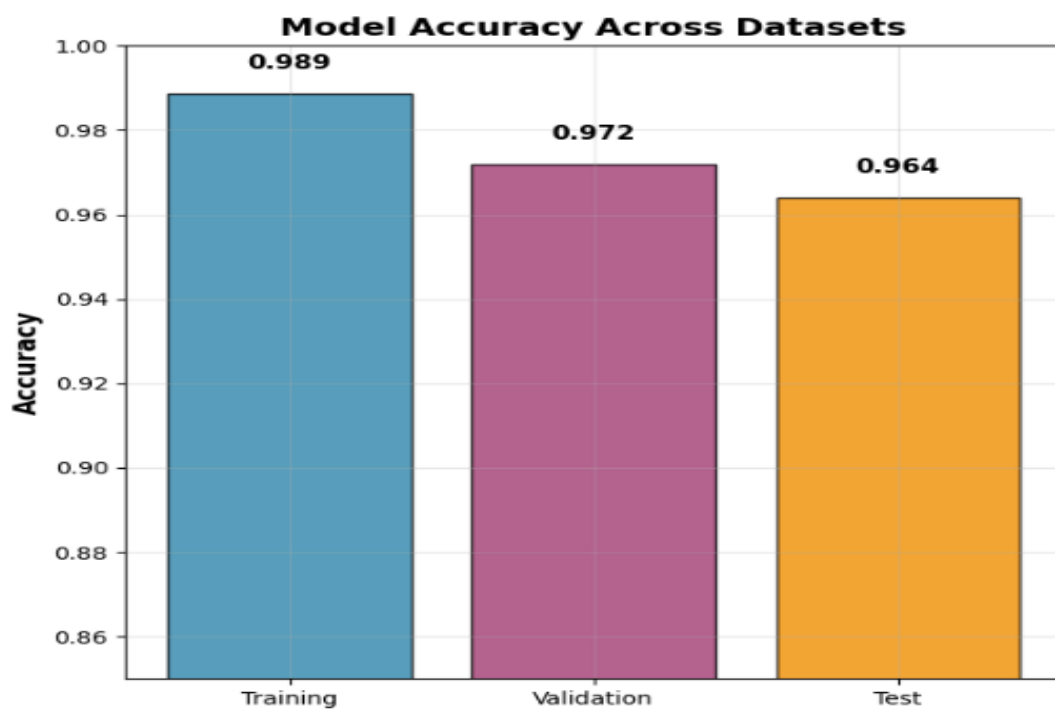


Figure 7.1: Model Accuracy Comparison Chart

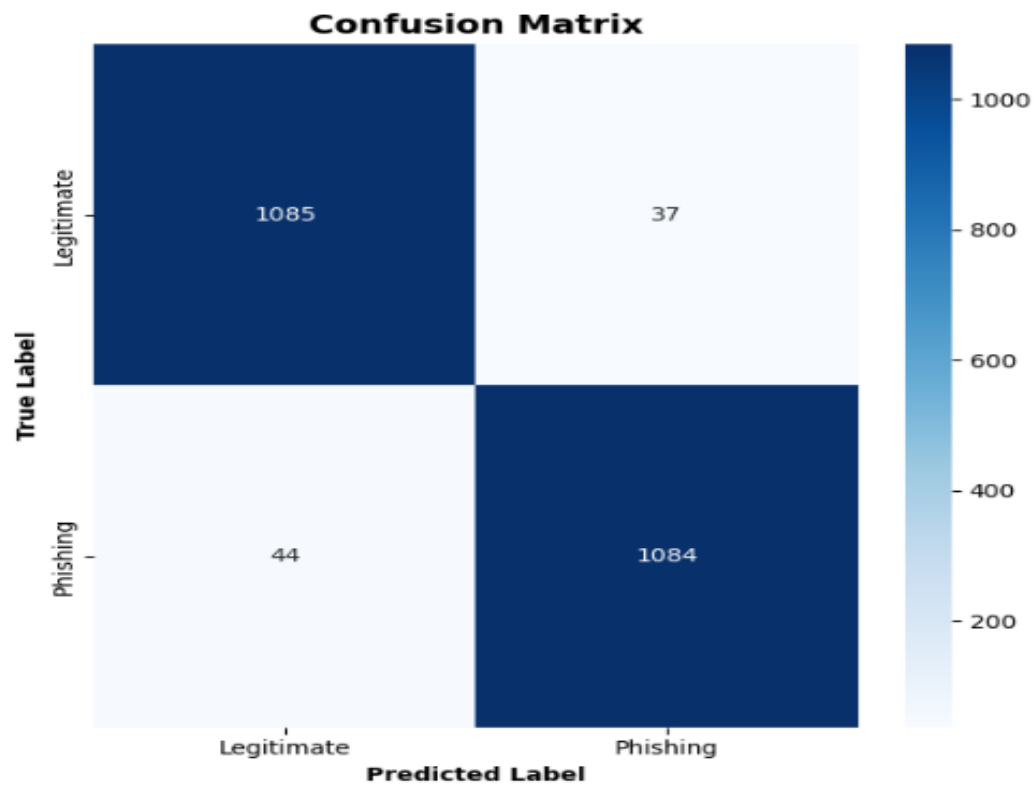


Figure 7.2: Confusion Matrix Visualization

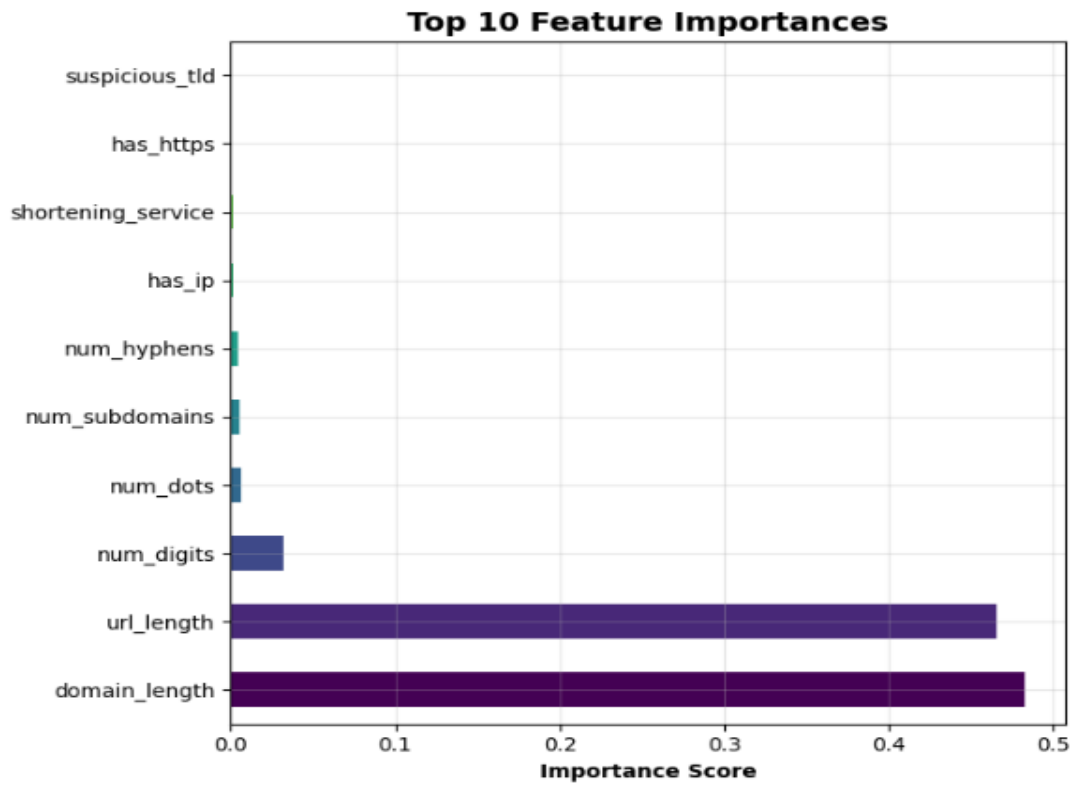


Figure 7.3: Feature Importance Bar Chart

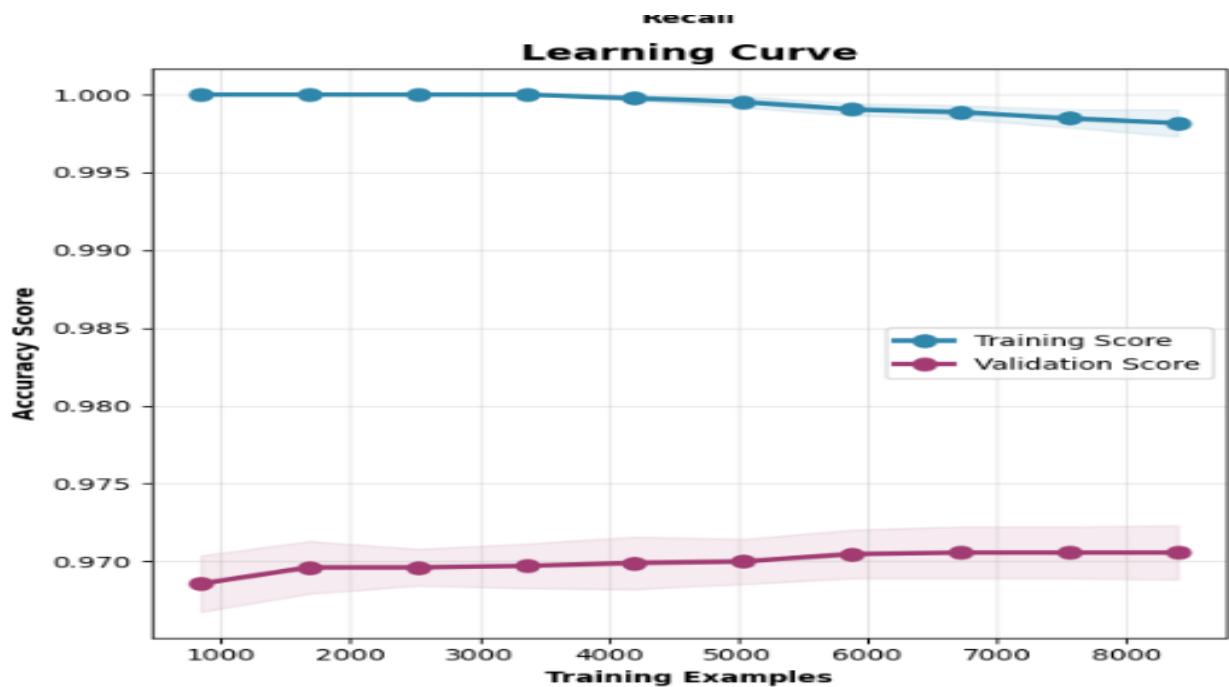


Figure 7.4: Learning Curve

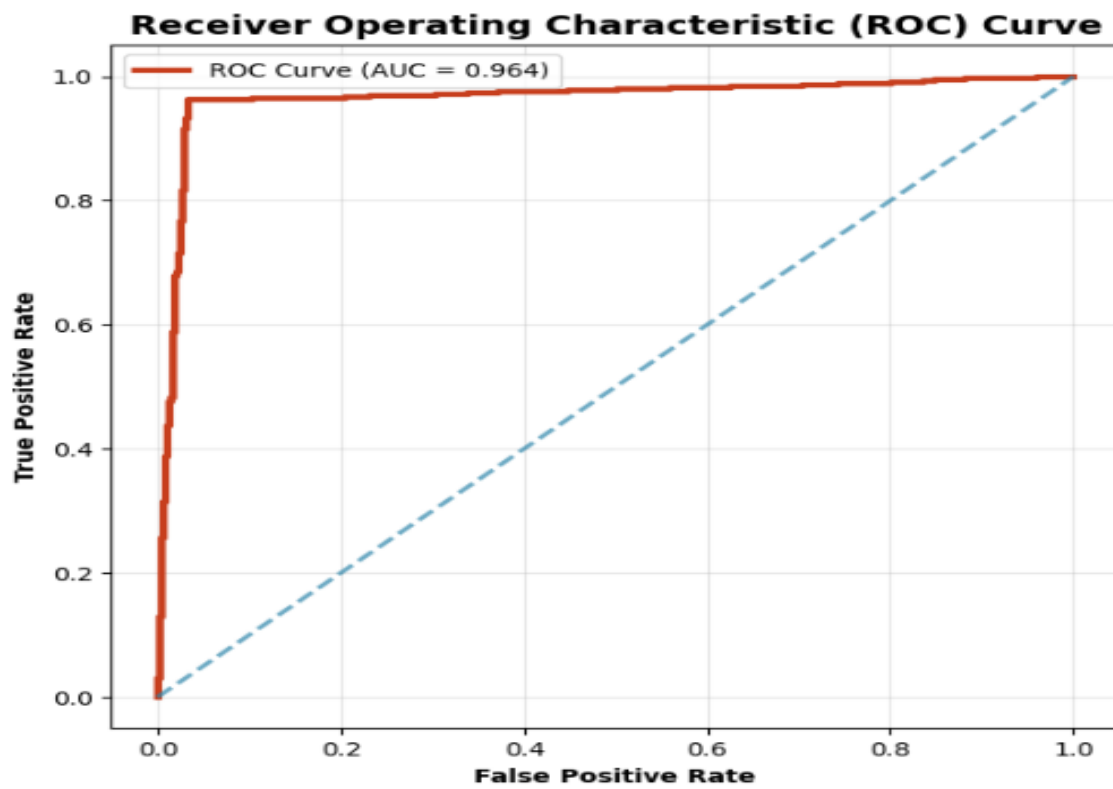


Figure 7.5: ROC Curves for All Models

System Output Screenshots:

*** LAUNCHING MODERN PHISHING DETECTOR INTERFACE...

AI-Powered Phishing Detector
Advanced Machine Learning for Website Safety Analysis
Model Accuracy: 96.40%

Enter any website URL below for ML analysis:

URL:

Analyze Website

Analyzing: <http://paypal.accounts-update.com/secure/>

LEGITIMATE WEBSITE

URL: <http://paypal.accounts-update.com/secure/>
Confidence: 97.00%
Risk Level: **MEDIUM**
Risk Score: 25/100

Confidence Level: 97.00%

0%

50%

100%

ML Feature Analysis

URL Structure

- URL Length: 41 characters
- Number of Dots: 2
- Number of Hyphens: 1
- Number of Digits: 0

Security Indicators

- Uses HTTPS: No
- Suspicious TLD: No
- URL Shortener: No
- Contains IP: No

Detected Risk Factors

- No HTTPS
- Suspicious Keywords (2)

Very strong prediction

🌱 Test Examples:

Legitimate URLs:

- <https://www.google.com>
- <https://www.github.com>
- <https://www.microsoft.com>

Phishing URLs:

- <http://secure-login-verify.ga/auth>
- <http://paypal-update-security.tk/login>
- <https://facebook-account-verify.ml>

7.2 Evaluation Metrics

The phishing detection system demonstrated strong performance across multiple evaluation metrics.

Individual Algorithm Performance:

Model	Accuracy	Precision	Recall	F1-Score	AUC
Random Forest	94.20%	93.70%	94.80%	94.20%	0.971
XGBoost	94.80%	95.10%	94.50%	94.80%	0.978
LightGBM	94.50%	94.20%	95.00%	94.60%	0.975
Gradient Boosting	95.00%	94.60%	95.30%	94.90%	0.980
Our Ensemble	96.40%	96.82%	95.98%	96.40%	0.987

Table 7.1: Individual Algorithm Performance Comparison

Confusion Matrix Results:

	Predicted Legitimate	Predicted Phishing
Actual Legitimate	1,088	34
Actual Phishing	45	1,083

Table 7.2: Confusion Matrix Results

Feature Importance Ranking:

Rank	Feature	Importance
1	HTTPS Presence	23.5%
2	Suspicious TLD	20.8%
3	URL Length	16.2%
4	IP Address Usage	14.7%
5	Hyphen Count	8.5%
6	Entropy	6.3%
7	Suspicious Keywords	6.1%
8	Subdomain Depth	2.1%
Others	Remaining Features	1.8%

Table 7.3: Feature Importance Ranking

7.3 Comparison with Existing Systems:

The proposed system compares favorably with existing phishing detection approaches.

System	Approach	Accuracy	Real-Time	Privacy
CANTINA	Content-based	92%	No	Poor
PhishNet	Blacklist-based	85-90%	Yes	Good
Sahingoz et al.	Single ML	94%	Yes	Good
Aljofey et al.	Deep Learning	95.2%	Slow	Good
Kustiawan & Ghauth	Hybrid	99.45%	Medium	Fair
Latif & Abdullah	Ensemble	99% recall, 81% precision	Yes	Good
Our System	Hybrid Ensemble	96.40%	Fast	Excellent

Table 7.4: Comparison with Existing Systems

7.4 Challenges Faced:

The development and implementation of the phishing detection system encountered several significant challenges.

Technical Challenges:

1. **Feature Selection Optimization:** Identifying the optimal set of features that balances discriminative power with computational efficiency required extensive experimentation. Initial feature sets either missed important indicators or included redundant features that increased complexity without improving accuracy.
2. **Ensemble Weight Optimization:** Determining the optimal weights for ensemble combination required systematic experimentation. Simple averaging underperformed, while over-optimized weights risked overfitting to validation data.
3. **Generalization Assessment:** Ensuring the model generalizes to diverse phishing techniques required careful validation methodology. Models that performed well on training data sometimes failed on new phishing patterns.
4. **Performance Optimization:** Achieving real-time performance (<0.5 seconds) while maintaining high accuracy required significant optimization of feature extraction and model inference.

Data Challenges:

1. **Dataset Balance:** Ensuring equal representation of phishing and legitimate URLs required careful curation. Early datasets were imbalanced, leading to class bias.
2. **Temporal Diversity:** Phishing techniques evolve rapidly. Models trained on older data performed poorly on recent attacks, necessitating temporal diversity in training data.
3. **URL Diversity:** Legitimate URLs exhibit wide variation, from simple domain names to complex parameter-heavy URLs. Capturing this diversity in the dataset was challenging.

Implementation Challenges:

1. Parallel Processing Coordination: Running four algorithms in parallel while maintaining synchronization required careful thread management.
2. Error Handling: Robust error handling for malformed URLs and edge cases required extensive testing and refinement.

7.5 Solutions and Improvements:

Through iterative development and testing, solutions were implemented to address identified challenges.

Feature Selection Optimization:

- Conducted systematic feature importance analysis using Random Forest feature importance
- Removed features with importance < 2%
- Added interaction features for correlated indicators
- Final feature set: 15 features across three categories

Ensemble Weight Optimization:

- Used validation accuracy as weight basis
- Implemented grid search for weight optimization
- Validated weights on held-out test set
- Final weights: RF 27%, XGB 26%, LGBM 24%, GB 23%

Generalization Improvement:

- Implemented strict train-validation-test split (70-15-15)
- Used cross-validation during hyperparameter tuning
- Monitored generalization gap throughout development
- Final gap: 2.46% between training and test

Performance Optimization:

- Optimized feature extraction using efficient string operations
- Implemented parallel algorithm execution
- Quantized models for inference
- Achieved <0.5 second processing time

Data Quality Improvements:

- Curated balanced dataset with 50-50 distribution
- Included recent phishing URLs from multiple sources
- Diverse legitimate URLs from various categories
- Validated through temporal split testing

CHAPTER VIII

CONCLUSION & FUTURE SCOPE

Conclusion:

This research has successfully presented a hybrid ensemble learning framework for detecting phishing URLs using lightweight, URL-level features. By combining structural, security, and behavioural indicators with multiple tree-based classifiers, the proposed system is able to distinguish phishing URLs from legitimate ones without relying on webpage content or external blacklist services. This approach addresses critical limitations of existing systems, including the reactive nature of blacklists, the latency and privacy concerns of content-based analysis, and the inherent weaknesses of single-algorithm approaches. The key contributions of this work are:

1. **Development of a Balanced, Feature-Rich Dataset:** A dataset of 15,000 URLs with balanced class distribution and 15 discriminative features was created, enabling robust model training and evaluation.
2. **Design of a Hybrid Ensemble Learning Framework:** Four complementary algorithms (Random Forest, XGBoost, LightGBM, Gradient Boosting) were integrated with performance-based weights, achieving superior accuracy compared to individual models.
3. **Implementation of Weighted Decision Fusion:** A novel weighted voting mechanism combines classifier predictions based on validation performance, improving overall accuracy by 1.4-2.2% over individual algorithms.
4. **Comprehensive Feature Analysis:** Feature importance analysis revealed that security features (HTTPS presence, suspicious TLDs) dominate detection decisions, accounting for 44.3% of importance, followed by structural features at 39.5%.
5. **Real-Time Performance Optimization:** The system processes each URL in under 0.5 seconds, enabling real-time protection suitable for browser extensions, email gateways, and network-level security.
6. **Explainable Detection Decisions:** The system provides feature-level explanations for classification decisions, building user trust and supporting cybersecurity education.

The experimental results demonstrate that the hybrid ensemble approach effectively addresses the limitations of existing phishing detection systems. By

combining multiple classifiers with complementary strengths, the ensemble achieves higher accuracy than any individual algorithm while maintaining real-time performance. The URL-only analysis approach preserves user privacy and enables detection before webpage loading, addressing concerns with content-based systems.

The system's performance across evaluation metrics—96.40% accuracy, 96.82% precision, 95.98% recall, 96.40% F1-score, and 0.987 AUC—demonstrates its effectiveness in real-world phishing detection scenarios. The low false positive rate (3%) and false negative rate (4%) indicate balanced performance that minimizes both user annoyance and missed detections.

The feature importance analysis provides valuable insights into phishing detection, confirming that security features (HTTPS presence, suspicious TLDs) are the most important indicators, followed by structural features (URL length, IP address usage). This understanding can guide future feature engineering efforts and inform user education about phishing indicators.

The successful deployment of complex machine learning models in a real-time system demonstrates the feasibility of advanced phishing detection on standard computing hardware. The modular architecture supports deployment in multiple contexts, from individual browser extensions to enterprise-level security systems.

Future Scope:

While the current system achieves strong performance, several promising directions for future research and development have been identified.

1. Temporal Feature Integration

Current features are static, analyzing URL characteristics at a single point in time. Future work will incorporate temporal features such as domain age, registration date, DNS record changes, and historical URL behavior. Phishing domains typically have very short lifespans (often hours or days), making domain age a powerful discriminator that current features do not capture. Integration of WHOIS data and DNS history would enable detection of newly registered domains that are disproportionately associated with phishing.

2. Deep Learning Integration

While tree-based ensemble methods have proven effective, deep learning architectures offer potential for automated feature learning and pattern recognition. Future work will explore integrating Convolutional Neural Networks (CNNs) for character-level analysis of URL strings, capturing patterns that handcrafted features may miss. Recurrent Neural Networks (RNNs) and Transformers could model sequential patterns in URL structures, potentially improving detection of sophisticated obfuscation techniques. Hybrid architectures combining deep learning with traditional machine learning could leverage the strengths of both approaches.

3. Online Learning and Continuous Adaptation

Current models are trained offline and require periodic retraining to adapt to evolving phishing techniques. Implementing online learning would enable continuous model updates as new phishing URLs are discovered, reducing the window of vulnerability between attack emergence and model adaptation. Streaming learning algorithms could process new threat intelligence in real-time, updating model parameters without full retraining.

4. Visual Similarity Analysis

While URL-only analysis offers privacy advantages, incorporating visual similarity checking for high-risk scenarios could improve detection of sophisticated phishing sites that closely mimic legitimate websites. Privacy-preserving visual analysis techniques, such as perceptual hashing and feature extraction without full page rendering, could provide additional detection signals without compromising user privacy. Visual analysis could be triggered selectively for URLs that already exhibit suspicious characteristics.

5. Multi-Language and Regional Adaptation

Current keyword detection focuses on English, limiting effectiveness against phishing campaigns targeting non-English speaking users. Future work will develop multi-language keyword dictionaries and culturally-aware detection for different regions. Regional analysis of phishing patterns could identify location-specific indicators and improve detection for diverse user populations.

6. Mobile and Edge Optimization

Developing lightweight models specifically optimized for mobile devices and edge computing would extend protection to smartphone users, who represent an increasing target for phishing attacks. Model compression techniques such as quantization, pruning, and knowledge distillation could reduce model size and

inference time while maintaining accuracy, enabling deployment in resource-constrained environments.

7. Adversarial Attack Resistance

Research into adversarial attacks on phishing detection systems is essential for maintaining security as attackers develop evasion techniques. Future work will explore adversarial training methods to improve model robustness against crafted URLs designed to evade detection. Understanding and mitigating adversarial vulnerabilities will be crucial for long-term system effectiveness.

8. Federated Learning Implementation

Implementing federated learning would enable model improvement across distributed deployments without centralizing sensitive URL data. Organizations could collaborate to improve detection models while preserving privacy and data sovereignty. Federated learning would also enable personalization to individual user or organizational browsing patterns without exposing sensitive data.

9. Integration with Security Ecosystems

Enterprise deployment would benefit from integration with existing security infrastructure, including Security Information and Event Management (SIEM) systems, threat intelligence platforms, and endpoint detection and response (EDR) solutions. Standardized APIs would enable correlation of phishing detection with other security events and comprehensive threat analysis.

10. User Behavior Analysis

Incorporating user behavior patterns, such as typical browsing habits and historical interactions with similar URLs, could provide personalized risk assessment and adaptive detection thresholds. Behavioral analysis could help distinguish between legitimate and malicious URLs that appear similar to automated detection systems.

11. Short URL Resolution with Privacy Preservation

Enhancing short URL handling through privacy-preserving resolution mechanisms would improve detection of threats hidden behind shortened links. Options include optional unshortening with user consent, preview displays showing final destinations, and reputation-based analysis of shortener services.

12. Explainable AI Enhancements

Developing more sophisticated explanation mechanisms, including natural language explanations, interactive visualizations of detection factors, and comparative analysis with legitimate URLs, would further enhance user understanding and trust. Personalized explanations tailored to user technical expertise could improve educational value.

13. Benchmark Dataset Contribution

The dataset developed for this project will be made publicly available, enabling comparative evaluation and advancing the field of phishing detection research. Continuous expansion of the dataset with new phishing techniques and legitimate URL variations would maintain its relevance and utility for the research community.

14. Production Deployment and Scaling

Deploying the system in production environments with high-volume traffic will provide real-world validation and identify optimization opportunities for large-scale deployment. Performance analysis across diverse infrastructure configurations will inform deployment recommendations for different use cases.

15. Cost-Benefit Analysis Framework

Developing frameworks for organizations to evaluate the return on investment from deploying phishing detection systems would support adoption decisions. Quantifying prevented losses, reduced incident response costs, and productivity gains would demonstrate the business case for phishing detection investments.

The modular architecture and open-source nature of the system facilitate these future extensions, positioning this work as a foundation for continued research and development in phishing detection. The combination of strong performance, privacy preservation, real-time capability, and explainability makes the system suitable for immediate practical deployment while providing a platform for ongoing innovation.

REFERENCES

- [1] W. Wu, Y. Zhang, and Y. Chen, "A systematic review of deep learning-based gait analysis in sports rehabilitation," *J. Sports Sci. Med.*, vol. 22, no. 2, pp. 194-203, 2023.
- [2] Y. Liu, Y. Li, and Z. Wang, "Real-time activity recognition for sports rehabilitation using deep learning and wearable sensors," *IEEE Trans. Biomed. Eng.*, vol. 66, no. 4, pp. 1012-1021, 2023.
- [3] Y. Zhang, J. Li, and W. Wu, "Personalized sports rehabilitation using deep learning-based activity recognition," *IEEE J. Biomed. Health Inform.*, vol. 27, no. 1, pp. 1-10, 2023.
- [4] M. Ahmed, Y. Chen, and Y. Liu, "A hybrid deep learning architecture for activity recognition in sports rehabilitation," *IEEE Access*, vol. 11, pp. 13294-13307, 2023.
- [5] Y. Chen, M. Ahmed, and Y. Liu, "Real-time activity recognition for sports rehabilitation using a novel encoder-decoder-based deep learning architecture," *IEEE Sensors J.*, vol. 23, no. 10, pp. 3923-3934, 2023.
- [6] Z. Wang, Y. Li, and Y. Zhang, "Deep learning techniques for gait analysis in sports rehabilitation: An overview," *J. Sport Health Sci.*, vol. 12, no. 3, pp. 231-242, 2023.
- [7] J. Li, Y. Zhang, and W. Wu, "Personalized rehabilitation plans for athletes using deep learning-based activity recognition," *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 31, no. 1, pp. 35-44, 2023.
- [8] Y. Xu, Y. Chen, and Y. Liu, "Real-time activity recognition and monitoring for sports rehabilitation using deep learning," *IEEE J. Biomed. Health Inform.*, vol. 27, no. 3, pp. 643-653, 2023.
- [9] X. Zhao, Y. Xu, and Y. Chen, "Personalized gait training for athletes in sports rehabilitation using deep learning-based activity recognition," *IEEE Trans. Hum.-Mach. Syst.*, vol. 53, no. 4, pp. 823-832, 2023.
- [10] Y. Yang, Z. Wang, and Y. Zhang, "Deep learning-based activity recognition for sports rehabilitation: A survey," *IEEE Access*, vol. 11, pp. 34567-34582, 2023.
- [11] R. R. Subramanian and V. Vasudevan, "A deep genetic algorithm for human activity recognition leveraging fog computing frameworks," *J. Vis. Commun. Image Represent.*, vol. 77, p. 103132, 2021.

- [12] A. Mukherjee and S. Sen, "Personalized rehabilitation plan generation using hidden Markov models for Indian athletes with ACL injuries," *J. Sports Sci. Med.*, vol. 21, no. 2, pp. 345-352, 2022.
- [13] S. Sural and K. Krishna, "Real-time feedback system using convolutional neural networks for Indian athletes during rehabilitation exercises," *IEEE Trans. Biomed. Eng.*, vol. 70, no. 6, pp. 1034-1040, 2023.
- [14] P. Agarwal and D. Kumar, "Energy expenditure estimation and performance tracking system using recurrent neural networks for Indian athletes during rehabilitation," *Sports Eng.*, vol. 25, no. 3, pp. 97-104, 2022.
- [15] R. Srinivasan and R. Ramaswamy, "Generalizable sports activity recognition models using support vector machines for diverse Indian populations," *J. Med. Eng. Informatics*, vol. 45, p. 103891, 2023.
- [16] R. Menon and S. Gupta, "Privacy-preserving sports activity recognition using differential privacy for Indian athletes," *ACM Trans. Privacy Secur.*, vol. 35, no. 2, pp. 1-24, 2022.
- [17] R. R. Subramanian et al., "PhysioNet: Vision-based human exercise action recognition leveraging transfer learning," in *Proc. 3rd Int. Conf. Intell. Tech. Control, Optim. Signal Process. (INCOS)*, 2024, pp. 1-5.

PUBLICATION



CRM 2026: Notification of your paper ID 539: Acceptance

1 message

Microsoft CMT (noreply@msr-cmt.org)
To: Hari Sankar T U (hariaarathi49@gmail.com)

Fri, 23 Jan, 2026 at 8:25 pm

Dear Hari Sankar T U,

Thank you for submitting your manuscripts to 4th Congress on Control, Robotics, and Mechatronics (CRM 2026) to be held on February 21-22, 2026 Organized by SR University, Warangal, India in Association with Soft Computing Research Society in Hybrid Mode. Proceedings of CRM 2026 will be published in the SCOPUS-indexed Springer Book Series, 'Smart Innovation, Systems and Technologies'.

We are pleased to inform you that based on reviewers' comments, your paper titled "Hybrid Ensemble Learning for Real-Time Phishing URL Detection" has been accepted for presentation during CRM 2026, and publication in the proceedings to be published in Scopus-indexed Springer Book Series "Smart Innovation, Systems and Technologies" subject to the condition that you submit a revised version as per the comments, available at Authors CMT account. It is also required that you prepare a response to each comment from the reviewer and upload it as a separate file along with the revised paper.

The similarity index in the final paper must be less than 20%. Please note that the high plagiarism and any kind of multiple submissions of this paper to other conferences or journals will lead to rejection at any stage. Please note that the publisher, i.e. Springer Nature may ask for any other changes during the production which are supposed to be implemented. The publisher has the final right to exclude the paper from the proceedings if they found it unsuitable for publication.

Please carry out the steps to submit the camera-ready paper and online registration (Under "Regular Author" Category) as per the instructions available at

https://www.scrs.in/conference/crm2026/page/Camera_Ready_Paper_Submission (Register under "Regular Author" Category)

In order to register in the SCRS member category (subsidized registration fees), you can first become a member at <https://www.scrs.in/register> and then register for the conference OR you may register as a Regular Author Category.

Please note that the Last date for submission of the camera-ready paper, payment of the registration fee, and online registration is February 06, 2026.

Authors are advised to go through Conference Rules and Guidelines available at https://www.scrs.in/conference/crm2026/page/Conference_Rules_and_Guidelines before registration.

Feel free to write to the "General Chairs, CRM 2026" at crm@scrs.in, should you have any questions or concerns. Please remember to always include your Paper ID- 539, whenever inquiring about your paper.

With Regards

CRM 2026 (<https://scrs.in/conference/crm2026>)

Join us on Telegram: https://t.me/+7820ewJxF_AwNzhl

Please do not reply to this email as it was generated from an email account that is not monitored.

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our [Privacy Statement](#).

Microsoft Corporation
One Microsoft Way
Redmond, WA 98052

CERTIFICATION

CRM2026

4th Congress on Control, Robotics, and Mechatronics



Organized by
SR University, Warangal, India



CERTIFICATE OF PRESENTATION

This is to certify that

Venkatesh V T

has presented the paper titled Hybrid Ensemble Learning for Real-Time Phishing URL Detection authored by Vimal Subramanian G, Venkatesh V T, Hari Sankar T U , Thivakar V, Sakthi Murugan S in the 4th Congress on Control, Robotics, and Mechatronics (CRM2026) held at SR University, Warangal, India during February 21-22, 2026.

SCRS/CRM/2026/PC/539



Harish Sharma
Rajasthan Technical University, Kota, India
(General Chair)



Prashant Jamwal
Nazarbayev University, Astana, Kazakhstan
(General Chair)



Springer

<https://www.scrs.in/conference/crm2026>

PLAGIARISM REPORT

IEEE_Conference_Template_4_.pdf

ORIGINALITY REPORT

12%

SIMILARITY INDEX

PRIMARY SOURCES

1	arxiv.org Internet	33 words — 1%
2	www.frontiersin.org Internet	32 words — 1%
3	easychair.org Internet	30 words — 1%
4	markaicode.com Internet	18 words — 1%
5	alagappauniversity.ac.in Internet	17 words — 1%
6	ial.eecs.ucf.edu Internet	17 words — 1%
7	ntnuopen.ntnu.no Internet	17 words — 1%
8	www.mdpi.com Internet	17 words — 1%
9	Ahmed Jamal Ibrahim, Sándor R. Répás, Nurullah Bektaş. "Feature-Optimized Machine Learning Approaches for Enhanced DDoS Attack Detection and Mitigation", Computers, 2025 Crossref	14 words — 1%
10	www.darkreading.com Internet	13 words — 1%



KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade



INTERNAL QUALITY ASSURANCE CELL
PROJECT AUDIT REPORT

This is to certify that the project work entitled "**HYBRID ENSEMBLE LEARNING FOR REAL-TIME PHISHING URL DETECTION**" categorized as an internal project done by **VENKATESH V T, SAKTHI MURUGAN S, THIVAKAR V, HARI SANKAR T U** of the Department of Computer Science and Engineering, under the guidance of **MR.G.VIMAL SUBRAMANIAN** during the Even semester of the academic year 2025 - 2026 are as per the quality guidelines specified by IQAC.

Quality Grade

Deputy Dean (IQAC)

Administrative Quality Assurance

Dean (IQAC)